

BRQC

Xinyi Huang Yongcheng Song* Rongmao Chen Jiang Zhang
Debiao He Xiaofeng Chen Junzuo Lai Yi Wang Chao Lin
Jiang Tan Kang Lin

June 30, 2026

BRQC is a code-based KEM running for standardization to ICCS’s NGCC competition in the category “public-key cryptographic algorithms”. Parameters sets are given for the three categories: 128, 256, and 512-bit security. The main features are:

- IND-CCA2 KEM
- Small public key size
- Null decryption failure rate
- Efficient implementations based on classical decoding algorithms
- Clean, clear, and flexible structure

Table 1: Sizes, DFR, classical/quantum security strength, and space complexity

Instance	sks	pks	cts	total	$ K $	DFR	Classical Security Strength	Quantum Security Strength	Space Complexity
BRQC-128	112	1954	3844	5798	64	Null	2^{181}	2^{90}	2^{181}
BRQC-256	126	3345	6626	9971	64	Null	2^{295}	2^{147}	2^{231}
BRQC-512	150	6562	13060	19622	64	Null	2^{566}	2^{283}	2^{278}

sks: the private key size. pks: the public key size. cts: the ciphertext size. total = pks + cts. $|K|$: the shared key size. All sizes are in bytes. DFR: decapsulation failure rate. The classical security strengths are determined by the MM attack (see Table 6). The quantum security strengths are the square root of the classical security strengths. The space complexity corresponds to the classical security strength.

*Contact: Yongcheng Song (yongchengsong@outlook.com)

Contents

1	Design Rationale and Changes	3
2	Notations and Preliminaries	3
2.1	Notations	3
2.2	Linear Codes with Rank Metric	4
2.3	Blockwise Rank Decoding Problem	4
2.4	Ideal Codes and Ideal BRSD Problem	6
2.5	Public Key Encryption	7
3	Specifications	8
3.1	Objects and Hash Functions	9
3.2	Gabidulin Codes and Decoding Algorithms	9
3.3	Public Key Encryption (BRQC.PKE)	12
3.4	Key Encapsulation Mechanism (BRQC.KEM)	14
4	Parameters	14
5	Performance	15
5.1	Reference Implementation	16
5.2	Optimized Implementation	16
6	Security Proof	16
7	Known Attacks on the BRSD Problem	19
7.1	Combinatorial Attacks	20
7.2	Algebraic Attacks	25
7.3	Resistance to Quantum Attacks	27
7.4	Estimated Security Strength	27
8	Side-Channel Attacks	29
9	Advantages and Limitations	29
	References	31

1 Design Rationale and Changes

Our BRQC (Blockwise RQC) is an improvement over RQC. The details of both algorithms and the changes made are as follows.

RQC: RQC (Rank Quasi-Cyclic) is a code-based cryptosystem based on rank-metric codes and designed as an IND-CCA2 secure KEM submitted to the NIST PQC standardization process in 2017. However, dedicated algebraic attacks [9–11], in particular the Maximal Minors (MM) attack, reduce its concrete security, and consequently RQC was not selected for the third round of the NIST PQC competition. To address this issue, new parameter sets [30] were proposed to provide sufficient resistance against algebraic attacks while maintaining competitive key sizes. Its main advantage is that its security relies entirely on the hardness of the random rank decoding problem and is independent of the structure of specific codes.

RQC mainly relies on the following errors, which tends to result in large parameter sizes:

- **(Non-)homogeneous errors.** These are introduced to ensure the hardness of the decoding problem and enhance the performance of RQC. However, their structure can lead to a rapid increase in the error weight during decryption, requiring the use of Gabidulin codes with large parameters for successful decoding.

BRQC: We introduce blockwise errors and construct BRQC in [40, 41]. This structure slows the growth of the error weight during decryption, enabling the use of error-correcting codes with smaller parameters. Compared to homogeneous errors, blockwise errors reduce the complexity of the decoding problem (by approximately a factor equal to the number of blocks in the exponent). Nevertheless, the gain in decoding capability outweighs the loss in complexity, resulting in an overall improvement in the parameters.

Our changes are also different in theory. The introduction of blockwise errors [41] brings the theory of sum-rank metric codes into code-based post-quantum cryptography. Blockwise errors are closely related to sum-rank metric errors [16, 27, 35], in which the error vector is partitioned into blocks, and the sum-rank weight is defined as the sum of the rank weights of each block. We here require that all blocks have direct-sum supports, specific weights, and general block length. The sum-rank metric forms a family of metrics encompassing both Hamming and rank metrics as special cases, and was originally introduced as the “extended rank metric” for multi-shot network coding in 2010 [31]. Sum-rank metric codes currently have independent interest. In particular, the work [16] established several fundamental results for sum-rank-metric matrix codes, including bounds, MacWilliams identities, and new code constructions. A key distinction between homogeneous rank errors and blockwise (sum-rank) errors is that the complexity of the associated decoding problems differs significantly, even when the total weight and code length are evenly divided [35, 41].

2 Notations and Preliminaries

In this section, we define some notations, and recall $\mathbb{F}_{q^m}$ -linear codes, the rank decoding problem, the ideal codes, and the ideal rank decoding problem.

2.1 Notations

- We denote by $\mathbb{N}$ the set of non-negative integer numbers, q prime or prime power, and $\mathbb{F}_{q^m}$ an extension of degree m of the finite field $\mathbb{F}_q$. We view $\mathbb{F}_{q^m}$ as an m -dimensional vector

space over $\mathbb{F}_q$. Let $\alpha \in \mathbb{F}_{q^m}$ be a primitive element such that $\boldsymbol{\alpha} = (1, \alpha, \dots, \alpha^{m-1}) \in \mathbb{F}_{q^m}^m$ is a basis of $\mathbb{F}_{q^m}$ over $\mathbb{F}_q$.

- Vectors (resp. matrices) are denoted by lower-case (resp. upper-case) bold letters.
- We use the notation $[N] := \{1, 2, \dots, N\}$ for the first N natural numbers. For two positive integers N_1 and N_2 ($N_1 \leq N_2$), the notation $\{N_1..N_2\}$ represents positive integers from N_1 to N_2 .
- The term $\text{negl}(\lambda)$ denotes a negligible function in a security parameter λ .
- Let $\mathcal{S}$ be a finite set or a distribution. $s \xleftarrow{\$} \mathcal{S}$ denotes that s is sampled from $\mathcal{S}$ by uniform and random way.
- Let $\mathcal{A}$ be an algorithm. We say that $\mathcal{A}$ is a PPT algorithm if it is a probabilistic polynomial-time algorithm.
- Let $\mathbf{G}$ be a game. $\mathbf{G} \Rightarrow b$ means that $\mathbf{G}$ outputs b .

2.2 Linear Codes with Rank Metric

Definition 2.1 (Rank Metric). For $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{F}_{q^m}^n$, each coordinate x_i is associated to a vector of $\mathbb{F}_q^m$ w.r.t. the basis $\boldsymbol{\alpha}$. Then $\mathbf{x}$ is associated to an $m \times n$ matrix given by $\text{Mat}(\mathbf{x}) = (x_{ij})_{i \in [m], j \in [n]}: \mathbf{x} = \boldsymbol{\alpha} \text{Mat}(\mathbf{x})$. The rank weight $\|\mathbf{x}\|_R$ of $\mathbf{x}$ is defined as the rank of $\text{Mat}(\mathbf{x})$. The rank distance $d_R(\mathbf{x}, \mathbf{y})$ between $\mathbf{x}$ and $\mathbf{y}$ in $\mathbb{F}_{q^m}^n$ is defined by $d_R(\mathbf{x}, \mathbf{y}) := \|\mathbf{x} - \mathbf{y}\|_R$.

Let $\langle x_1, x_2, \dots, x_n \rangle_{\mathbb{F}_q}$ be the $\mathbb{F}_q$ -linear subspace of $\mathbb{F}_{q^m}^n$ generated by linear combinations over $\mathbb{F}_q$ of coordinates of $\mathbf{x}$. The support $\text{Supp}(\mathbf{x})$ of $\mathbf{x}$ is defined as the $\mathbb{F}_q$ -linear subspace $\langle x_1, x_2, \dots, x_n \rangle_{\mathbb{F}_q}$, i.e., $\text{Supp}(\mathbf{x}) = \langle x_1, x_2, \dots, x_n \rangle_{\mathbb{F}_q}$. It follows from definition that $\|\mathbf{x}\|_R = \dim(\text{Supp}(\mathbf{x}))$. The set of such errors of weight r and length n is denoted by $\mathcal{S}_r^n$.

Support and Coefficient Matrices of the Error. For an error $\mathbf{e} \in \mathcal{S}_r^n$, let $\boldsymbol{\varepsilon} = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_r) \in \mathbb{F}_{q^m}^r$ be a basis of $\text{Supp}(\mathbf{e})$, then there exists a matrix $\mathbf{C} \in \mathbb{F}_q^{r \times n}$ of rank r such that $\mathbf{e} = \boldsymbol{\varepsilon} \mathbf{C}$. Under the basis $\boldsymbol{\alpha}$, there exists a matrix $\mathbf{S} \in \mathbb{F}_q^{m \times r}$ of rank r such that $\boldsymbol{\varepsilon} = \boldsymbol{\alpha} \mathbf{S}$. Then $\mathbf{e} = \boldsymbol{\alpha} \mathbf{S} \mathbf{C}$. We call $\mathbf{S}$ and $\mathbf{C}$ respectively support matrix and coefficient matrix.

Definition 2.2 ($\mathbb{F}_{q^m}$ -Linear Codes with Rank Metric). An $\mathbb{F}_{q^m}$ -linear code embedded with rank metric of length n and dimension k is a subspace of dimension k of $\mathbb{F}_{q^m}^n$. Such $\mathbb{F}_{q^m}$ -linear codes are denoted by $[n, k]_{q^m}$.

For an $[n, k]_{q^m}$ -linear code $\mathcal{C}$, a matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ is called *generator matrix* iff $\mathcal{C} = \{\mathbf{m} \mathbf{G} : \mathbf{m} \in \mathbb{F}_{q^m}^k\}$ and a matrix $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$ is called *parity-check matrix* iff $\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_{q^m}^n : \mathbf{H} \mathbf{c}^\top = \mathbf{0}\}$. The systematic forms of $\mathbf{G}$ and $\mathbf{H}$ are respectively defined as $[\mathbf{I}_k \ \mathbf{P}]$ and $[-\mathbf{P}^\top \ \mathbf{I}_{n-k}]$ where $\mathbf{P} \in \mathbb{F}_{q^m}^{k \times (n-k)}$.

2.3 Blockwise Rank Decoding Problem

The security of code-based cryptosystems is based on the hardness of decoding assumptions. The decoding assumptions can be expressed by the generator and parity-check matrices, respectively. In the rank metric setting, we call the Rank Decoding (RD) problem and the Rank Syndrome Decoding (RSD) problem, respectively. Their computational versions are as follows.

Rank Decoding (RD) Problem: Let $\mathbf{G}$ be a generator matrix of a random $[n, k]_{q^m}$ -linear code $\mathcal{C}$, $\mathbf{y} \in \mathbb{F}_{q^m}^n$, and $r \in \mathbb{N}$. The problem is to find $\mathbf{x} \in \mathbb{F}_{q^m}^k$ and $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{y} = \mathbf{x}\mathbf{G} + \mathbf{e}$.
Rank Syndrome Decoding (RSD) Problem: Let $\mathbf{H}$ be a parity-check matrix of a random $[n, k]_{q^m}$ -linear code $\mathcal{C}$, $\mathbf{s} \in \mathbb{F}_{q^m}^{n-k}$, and $r \in \mathbb{N}$. The problem is to find $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{s} = \mathbf{H}\mathbf{e}^\top$.

The hardness of two problems is equivalent. Even if the RD problem is not known to be NP-complete, there is a randomized reduction [22] from the RD problem to an NP-complete problem, namely to decoding in the Hamming metric [12]. An RD instance can also be viewed as a structured MinRank instance; more precisely, there exists a general reduction [19] from any RD instance to the MinRank instance. The MinRank problem was proven to be NP-complete in [15].

The Blockwise Rank Decoding (BRD) problem [41] is a variant of the RD problem and is proposed to improved cryptosystems. The difference mainly consists in the definition of the error $\mathbf{e}$. In the BRD problem, one finds a blockwise error.

Let $\ell \in \mathbb{N}$. Let $\mathbf{n} = (n_1, n_2, \dots, n_\ell) \in \mathbb{N}^\ell$ and $\mathbf{r} = (r_1, r_2, \dots, r_\ell) \in \mathbb{N}^\ell$. Let $n = \sum_{i=1}^\ell n_i$ and $r = \sum_{i=1}^\ell r_i$.

Definition 2.3 (Blockwise Errors [41]). Let $\mathbf{e}_i \in \mathbb{F}_{q^m}^{n_i}$ be a vector of weight r_i for $i \in [\ell]$. An error $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell) \in \mathbb{F}_{q^m}^n$ is called a blockwise error if the supports of sub-vectors $\mathbf{e}_i$'s are in direct sum.

Let $\mathcal{S}_r^n$ denote the set of such blockwise errors. Since $\mathbf{e}_i$'s supports are in direct sum iff $\dim\left(\sum_{i=1}^\ell \text{Supp}(\mathbf{e}_i)\right) = \sum_{i=1}^\ell \dim(\text{Supp}(\mathbf{e}_i))$, we have

$$\mathcal{S}_r^n = \left\{ \mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell) : \mathbf{e}_i \in \mathcal{S}_{r_i}^{n_i}, \dim\left(\sum_{i=1}^\ell \text{Supp}(\mathbf{e}_i)\right) = \sum_{i=1}^\ell \dim(\text{Supp}(\mathbf{e}_i)) \right\}.$$

$$\|\mathbf{e}\|_R = \dim(\text{Supp}(\mathbf{e})) = \dim\left(\sum_{i=1}^\ell \text{Supp}(\mathbf{e}_i)\right) = \sum_{i=1}^\ell \dim(\text{Supp}(\mathbf{e}_i)) = \sum_{i=1}^\ell r_i = r.$$

Remark 2.1. We note that the work [40] introduced the notion of blockwise errors. The supports of $\mathbf{e}_i$'s are required to be mutually disjoint. This condition gives rise to two drawbacks:

First, this condition is not sufficient to guarantee that the rank weight equals r . The rank weight of $\mathbf{e}$ only satisfies

$$\|\mathbf{e}\|_R = \dim(\text{Supp}(\mathbf{e})) = \dim\left(\sum_{i=1}^\ell \text{Supp}(\mathbf{e}_i)\right) \leq \sum_{i=1}^\ell \dim(\text{Supp}(\mathbf{e}_i)) = \sum_{i=1}^\ell r_i = r.$$

Second, this condition in fact will render both the support/coefficient matrices of the error highly ambiguous and inherently complex, thereby hindering a rigorous complexity analysis of the underlying decoding problem.

In [41], the authors updated the condition and required that the supports of $\mathbf{e}_i$'s form a direct sum. Under this stronger condition, the rank weight of $\mathbf{e}$ is guaranteed to be equal to r , and the support/coefficient matrices are also clear, which facilitates a rigorous complexity analysis on the BRD problem.

Nevertheless, this refinement does not diminish the relevance of “mutually disjoint” for optimizing cryptosystems. For the purpose of optimizing cryptosystems, “mutually disjoint” condition is already sufficient in practice.

Definition 2.4 (Blockwise RD (BRD) Problem [41]). Let $\mathbf{G}$ be a generator matrix of a random $[n, k]_{q^m}$ -linear code $\mathcal{C}$ and $\mathbf{y} \in \mathbb{F}_{q^m}^n$. The problem is to find a $\mathbf{x} \in \mathbb{F}_{q^m}^k$ and a blockwise error $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{y} = \mathbf{x}\mathbf{G} + \mathbf{e}$.

Definition 2.5 (Blockwise RSD (BRSD) Problem [41]). Let $\mathbf{H}$ be a parity-check matrix of a random $[n, k]_{q^m}$ -linear code $\mathcal{C}$ and $\mathbf{s} \in \mathbb{F}_{q^m}^{n-k}$. The problem is to find $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{s} = \mathbf{H}\mathbf{e}^\top$.

The variants are exactly the RSD and RD problems when $\ell = 1$. We denote them by $\text{BRD}_{q,m,n,k,r}^{\ell,n,r}$ and $\text{BRD}_{q,m,n,k,r}^{\ell,n,r}$, respectively. The randomized reduction from the BRD problem to the NP-complete Hamming metric decoding problem [12] can refer to [35]. Any BRD instance can also be easily reduced to the MinRank instance by adapting the idea of [19]. We use the decisional BRSD problem for the security proof of cryptosystems.

Definition 2.6 (Decisional BRSD (DBRSD) Problem). Let $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k) \times n}$ be a parity-check matrix of a random $[n, k]_{q^m}$ -linear code. The DBRSD problem is hard, if for any PPT adversary $\mathcal{B}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{B}}^{\text{DBRSD}_{q,m,n,k,r}^{\ell,n,r}}(\lambda) := \left| \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{s}) = 1 : \mathbf{s} \xleftarrow{\$} \mathbb{F}_{q^m}^{n-k} \right] - \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{H}\mathbf{e}^\top) = 1 : \mathbf{e} \leftarrow \mathcal{S}_r^n \right] \right|.$$

The best known attacks on the decisional version remain the direct attacks on the computational version.

2.4 Ideal Codes and Ideal BRSD Problem

In this section, we first recall the definitions of ideal codes, then give the ideal variants of the BRSD problem. Let $P(X)$ be a polynomial of degree n in $\mathbb{F}_q[X]$ and $\mathcal{R} = \mathbb{F}_{q^m}[X]/\langle P(X) \rangle$. By the map $\psi : \mathbb{F}_{q^m}^n \rightarrow \mathcal{R}$, $\mathbb{F}_{q^m}^n$ is isomorphic to $\mathcal{R}$. Let $\mathbf{u}(X) = \sum_{i=0}^{n-1} u_i X^i \in \mathcal{R}$ be the polynomial associated the vector $\mathbf{u} = (u_0, u_1, \dots, u_{n-1}) \in \mathbb{F}_{q^m}^n$. The vector $(0, 0, \dots, 0) \in \mathbb{F}_{q^m}^n$ corresponds to the zero $\mathbf{0}$ of $\mathcal{R}$. The vector $(1, 0, \dots, 0) \in \mathbb{F}_{q^m}^n$ corresponds to the identity $\mathbf{1}$ of $\mathcal{R}$. For $\mathbf{u} \in \mathbb{F}_{q^m}^n$ and $\mathbf{v} \in \mathbb{F}_{q^m}^n$, the product $\mathbf{u} \cdot \mathbf{v}$ is defined as the vector of coefficients of $\mathbf{u}(X)\mathbf{v}(X) \bmod P(X)$, i.e., $\psi^{-1}(\mathbf{u}(X)\mathbf{v}(X) \bmod P(X))$.

Definition 2.7 (Ideal Matrix). The ideal matrix generated by $\mathbf{v} \in \mathbb{F}_{q^m}^n$ is defined as an $n \times n$ matrix:

$$\mathcal{IM}(\mathbf{v}) = \begin{bmatrix} \psi^{-1}(\mathbf{v}(X)) \\ \psi^{-1}(X\mathbf{v}(X) \bmod P(X)) \\ \vdots \\ \psi^{-1}(X^{n-1}\mathbf{v}(X) \bmod P(X)) \end{bmatrix}.$$

With the ideal matrix, the product $\mathbf{u} \cdot \mathbf{v}$ is equivalent to the vector-matrix product

$$\begin{aligned} \mathbf{u} \cdot \mathbf{v} &= \psi^{-1}(\mathbf{u}(X)\mathbf{v}(X) \bmod P(X)) = \psi^{-1} \left(\sum_{i=0}^{n-1} u_i X^i \mathbf{v}(X) \bmod P(X) \right) \\ &= \psi^{-1} \left(\sum_{i=0}^{n-1} u_i (X^i \mathbf{v}(X) \bmod P(X)) \right) = (u_0, \dots, u_{n-1}) \mathcal{IM}(\mathbf{v}) = \mathbf{u} \mathcal{IM}(\mathbf{v}). \end{aligned}$$

Thus, $\mathbf{u} \cdot \mathbf{v} = \mathbf{u} \mathcal{IM}(\mathbf{v}) = \mathbf{v} \mathcal{IM}(\mathbf{u}) = \mathbf{v} \cdot \mathbf{u}$. It is clear that, for any $\mathbf{h}_1, \mathbf{h}_2, \mathbf{e}_1$, and $\mathbf{e}_2 \in \mathbb{F}_{q^m}^n$,

$$\mathbf{h}_1 \cdot \mathbf{e}_1 + \mathbf{h}_2 \cdot \mathbf{e}_2 = \mathbf{s} \iff \begin{bmatrix} \mathbf{h}_1 & \mathbf{h}_2 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \end{bmatrix} = \mathbf{s} \iff \begin{bmatrix} \mathcal{IM}(\mathbf{h}_1)^\top & \mathcal{IM}(\mathbf{h}_2)^\top \end{bmatrix} \begin{bmatrix} \mathbf{e}_1^\top \\ \mathbf{e}_2^\top \end{bmatrix} = \mathbf{s}^\top$$

To reduce the size of rank-based cryptosystems, the family of ideal codes is introduced in rank-based cryptography. The ideal codes are codes with a systematic generator matrix consisting of blocks of ideal matrices.

Definition 2.8 (Ideal Codes [4, 6]). Let $P(X)$ be a polynomial of degree n in $\mathbb{F}_q[X]$. An $[\ell n, nt]_{q^m}$ code $\mathcal{C}$ is an (ℓ, t) -ideal code if its generator matrix under the systematic form is of the form

$$\mathbf{G} = \begin{bmatrix} \mathcal{IM}(\mathbf{g}_{1,1}) & \dots & \mathcal{IM}(\mathbf{g}_{1,\ell-t}) \\ \mathbf{I}_{tn} & \vdots & \ddots & \vdots \\ & \mathcal{IM}(\mathbf{g}_{t,1}) & \dots & \mathcal{IM}(\mathbf{g}_{t,\ell-t}) \end{bmatrix}$$

where $(\mathbf{g}_{i,j})_{i \in [t], j \in [\ell-t]}$ are vectors of $\mathbb{F}_{q^m}^n$.

We only consider $t = 1$ and $[\ell n, n]_{q^m}$ -ideal codes. An $[\ell n, n]_{q^m}$ -ideal code has the systematic parity-check matrix in the form of

$$\mathbf{H} = \begin{bmatrix} & \mathcal{IM}(\mathbf{h}_1)^\top \\ \mathbf{I}_{(\ell-1)n} & \vdots \\ & \mathcal{IM}(\mathbf{h}_{\ell-1})^\top \end{bmatrix} \in \mathbb{F}_{q^m}^{(\ell-1)n \times \ell n} \text{ or } \begin{bmatrix} \mathbf{1} & \dots & \mathbf{0} & \mathbf{h}_1 \\ \mathbf{1} & \dots & \mathbf{0} & \mathbf{h}_2 \\ \vdots & \ddots & \vdots & \vdots \\ \mathbf{0} & \dots & \mathbf{1} & \mathbf{h}_{\ell-1} \end{bmatrix} \in \mathcal{R}^{(\ell-1) \times \ell}.$$

Definition 2.9 (Ideal BRSD (IBRSD) Problem). Let $\mathbf{H}$ be a systematic parity-check matrix of a random $[\ell n, n]_{q^m}$ -ideal code and $\mathbf{s} \in \mathbb{F}_{q^m}^{(\ell-1)n}$. The problem is to find a blockwise error $\mathbf{e} \in \mathcal{S}_r^n$ such that $\mathbf{s} = \mathbf{H}\mathbf{e}^\top$.

This is called the computational IBRSD problem. We base the security of cryptosystems on its decisional version.

Definition 2.10 (Decisional IBRSD (DIBRSD) Problem). Let $\mathbf{H}$ be a systematic parity-check matrix of a random $[\ell n, n]_{q^m}$ -ideal code. The DIBRSD is hard, if for any PPT adversary $\mathcal{B}$, the following advantage is negligible:

$$\text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,\ell n,n,r}^{\ell,n,r}}(\lambda) := \left| \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{s}) = 1 : \mathbf{s} \xleftarrow{\$} \mathbb{F}_{q^m}^{(\ell-1)n} \right] - \Pr \left[\mathcal{B}(\mathbf{H}, \mathbf{H}\mathbf{e}^\top) = 1 : \mathbf{e} \xleftarrow{\$} \mathcal{S}_r^n \right] \right|.$$

As hardness assumptions [4, 29] of the ideal versions of the RSD problem, we have

- The IBRSD problem is as hard as the BRSD problem and there is no known significant improvement by exploiting the ideal structure, typically choosing a $P(X)$ with many small factors to resist the folding attack [26], and in our case, $P(X)$ is set as an irreducible polynomial.
- The attacks on the DIBRSD problem remain the direct attacks on the IBRSD problem, thus DIBRSD and IBRSD have an equivalent complexity.

2.5 Public Key Encryption

Definition 2.11 (Public Key Encryption). A Public Key Encryption (PKE) scheme $\text{PKE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$ consists of four polynomial-time algorithms:

- $\text{Setup}(1^\lambda) \rightarrow \text{par}$: The setup algorithm takes the security parameter 1^λ as input and outputs global system parameters par . We assume that par implicitly defines a plaintext space $\mathcal{M} = \mathcal{M}_{\text{par}}$.

- $\text{KGen}(\text{par}) \rightarrow (\text{pk}, \text{sk})$: The key generation algorithm that takes par as input and outputs a public key pk and a private key sk .
- $\text{Enc}(\text{pk}, \text{m}; \theta) \rightarrow \text{ct}$: The encryption algorithm that takes pk , a plaintext $\text{m} \in \mathcal{M}$, and randomness θ as inputs and outputs a ciphertext ct .
- $\text{Dec}(\text{sk}, \text{ct}) \rightarrow \text{m} / \perp$: The decryption algorithm that takes sk and ct as inputs and outputs $\text{m} \in \mathcal{M}$ or a rejection symbol $\perp$.

Correctness. The PKE is $(1 - \delta)$ -correct, if for any $\text{par} \leftarrow \text{Setup}(1^\lambda)$ and any $\text{m} \in \mathcal{M}$, the following probability holds:

$$\Pr \left[\text{m} = \text{m}' : (\text{pk}, \text{sk}) \leftarrow \text{KGen}(\text{par}), \text{ct} \leftarrow \text{Enc}(\text{pk}, \text{m}; \theta), \text{m}' \leftarrow \text{Dec}(\text{sk}, \text{ct}) \right] \geq 1 - \delta,$$

where the probability depends on the randomness of KGen and Enc , and δ is called the decryption failure rate.

Security Model. The basic security requirement for PKE is IND-CPA, Indistinguishability under Chosen Plaintext Attacks. An IND-CPA secure PKE is defined by the game $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda)$ between a challenger and a PPT adversary $\mathcal{A}$:

Game $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda)$
01 $\text{par} \leftarrow \text{Setup}(1^\lambda)$
02 $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(\text{par})$
03 $(\text{m}_0, \text{m}_1) \xleftarrow{\$} \mathcal{A}(\text{par}, \text{pk})$
04 $b \xleftarrow{\$} \{0, 1\}$
05 $\text{ct} \leftarrow \text{Enc}(\text{pk}, \text{m}_b; \theta)$
06 $b' \leftarrow \mathcal{A}(\text{par}, \text{pk}, \text{ct})$
07 $b' \neq b : \text{return } 0$
08 return 1

Figure 1: The IND-CPA security game for PKE $\text{PKE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$

Definition 2.12 (IND-CPA Security). A PKE scheme $\text{PKE} = (\text{Setup}, \text{KGen}, \text{Enc}, \text{Dec})$ is IND-CPA secure if for any PPT adversary $\mathcal{A}$, its advantage is bounded by a negligible function $\text{negl}(\lambda)$:

$$\text{Adv}_{\text{PKE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \left| \Pr [\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda).$$

3 Specifications

In this section, we describe our scheme: BRQC. We begin with the PKE version (BRQC.PKE), then exploit the salted Fujisaki–Okamoto (FO) transformation [25] to obtain an IND-CCA2 KEM (BRQC.KEM).

3.1 Objects and Hash Functions

Field elements. Elements of $\mathbb{F}_{q^m}$ are represented as vectors of size m over $\mathbb{F}_q$. In our setting, q is set to 2. Thus, any $e \in \mathbb{F}_{q^m}$ can be represented as $(e_0, e_1, \dots, e_{m-1}) \in \mathbb{F}_q^m$.

Vectors. Elements of $\mathbb{F}_{q^m}^n$ are represented as n -dimensional arrays over $\mathbb{F}_{q^m}$.

Polynomials. When $\mathbf{e}$ is viewed as a polynomial in $\mathcal{R}$, its first coordinate corresponds to the constant coefficient of $\mathbf{e}$.

Randomness. Application randomness is drawn from the deterministic random bit generator provided by the NGCC API, which is instantiated with SM3 and maintains a 55-byte internal state.

Seeds. The considered `seedexpander` has been provided by the NGCC. It is initialized with a byte string of length 64 and used to deterministically derive the key pair and the encryption randomness from a master seed.

Salts. A fresh salt is drawn in every `Encaps` call as a bit string of length 512, and its length is denoted by `len`. The salt is bound to the ciphertext to prevent multi-target attacks.

Hash Functions. The hash functions `G` and `H` are instantiated using the `pseudohash` construction provided by the NGCC API (built from SM3 and HMAC-SM3) with a 64-byte output. The output of `G` is used as a seed, and the output of `H` is the encapsulated key.

3.2 Gabidulin Codes and Decoding Algorithms

Gabidulin codes are rank metric codes and were proposed by Gabidulin [20]. A Gabidulin code is the evaluation of q -polynomials [32]. The q -polynomial is essentially the polynomial of the Frobenius automorphism over $\mathbb{F}_{q^m}$. We here recall the Frobenius automorphism from $\mathbb{F}_{q^m}$ to $\mathbb{F}_{q^m}$ for facilitating the understanding of the essence of q -polynomials. We denote it as:

$$\begin{aligned} \theta : \mathbb{F}_{q^m} &\longrightarrow \mathbb{F}_{q^m} \\ x &\longmapsto \theta(x) := x^q. \end{aligned}$$

The Frobenius automorphism θ is an $\mathbb{F}_q$ -automorphism of $\mathbb{F}_{q^m}$. We use its following properties:

- $\forall x_1, x_2 \in \mathbb{F}_{q^m}, \beta_1, \beta_2 \in \mathbb{F}_q$: $\theta(x_1 x_2) = \theta(x_1) \theta(x_2)$, $\theta(\beta_1 x_1 + \beta_2 x_2) = \beta_1 \theta(x_1) + \beta_2 \theta(x_2)$ (linearity over $\mathbb{F}_q$),
- $\forall i \in \mathbb{N}$, $\theta^i(x) = \theta^{i-1}(\theta(x)) = \theta^{i-1}(x^q) = \dots = \theta(x^{q^{i-1}}) = x^{q^i}$,
- $\forall i \in \mathbb{N}$, θ^i is also a Frobenius automorphism. This is because the set of all Frobenius automorphisms is a cyclic group of order m (The identity is θ^0).

Let $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{F}_{q^m}^n$. The Frobenius automorphism θ can be applied to $\mathbf{x}$ coordinate-wise: $\theta(\mathbf{x}) = (\theta(x_1), \theta(x_2), \dots, \theta(x_n)) = (x_1^q, x_2^q, \dots, x_n^q)$. Further, for any $i \in \mathbb{N}$, $\theta^i(\mathbf{x}) = (\theta^i(x_1), \theta^i(x_2), \dots, \theta^i(x_n)) = (x_1^{q^i}, x_2^{q^i}, \dots, x_n^{q^i})$.

Definition 3.1 (q -Polynomials [32]). A q -polynomial of q -degree n over $\mathbb{F}_{q^m}$ is a polynomial of the form $f(x) = \sum_{i=0}^n \theta^i(x) = \sum_{i=0}^n f_i x^{q^i}$ for $f_i \in \mathbb{F}_{q^m}$, $f_n \neq 0$.

A q -polynomial in a variable x is essentially a polynomial of θ acting on x . The addition (resp. symbolic product “ $\circ$ ”) of two q -polynomials corresponds to the addition (resp. composition “ $\bullet$ ”) of two polynomials of θ . Let $a(x) = \sum_i a_i x^{q^i}$ and $b(x) = \sum_i b_i x^{q^i} \in \mathcal{L}[x]$. Let $a(\theta) = \sum_i a_i \theta^i$ and $b(\theta) = \sum_i b_i \theta^i$.

- $a(\theta)(x) = (\sum_i a_i \theta^i)(x) = \sum_i a_i \theta^i(x) = \sum_i a_i x^{q^i} = a(x)$,
- $(a(\theta) + b(\theta))(x) = (\sum_i (a_i + b_i) \theta^i)(x) = \sum_i (a_i + b_i) x^{q^i} = a(x) + b(x)$,
- $(a(\theta) \bullet b(\theta))(x) = a(b(\theta))(x) = a(b(x)) = a(x) \circ b(x)$.

We denote the q -degree of a q -polynomial $f(x)$ by $\deg_q f(x)$. Let $\mathcal{L}[x]$ be the set of q -polynomials and $\mathcal{L}_{\leq r}[x]$ be the set of q -polynomials of q -degree $\leq r$. Let $a(x) = \sum_i a_i x^{q^i}$ and $b(x) = \sum_i b_i x^{q^i} \in \mathcal{L}[x]$:

- The addition “ $+$ ” of $a(x)$ and $b(x)$: $a(x) + b(x) = \sum_i (a_i + b_i) x^{q^i}$,
- The symbolic product “ $\circ$ ” of $a(x)$ and $b(x)$: $a(x) \circ b(x) = a(b(x))$.

It is clear that the set of q -polynomials together with addition and symbolic product forms a noncommutative ring. In such a ring, the symbolic product is associative and distributive w.r.t. both right and left product. The identity element is $\theta^0(x) = x^{q^0} = x$ and there are no divisors of zero, i.e., $a(x) \circ b(x) = 0$ implies $a(x) = 0$ or $b(x) = 0$. A q -polynomial $c(x)$ is said to be symbolically right divisible by $b(x)$ if $c(x) = a(x) \circ b(x)$. When $c(x) = a(x) \circ b(x)$, we say that $c(x)$ is symbolically left divisible by $a(x)$, which is denoted by $b(x) = a(x) \setminus c(x)$.

A q -polynomial $f \in \mathcal{L}[x]$ satisfies:

- $\forall x_1, x_2 \in \mathbb{F}_{q^m}, \beta_1, \beta_2 \in \mathbb{F}_q, f(\beta_1 x_1 + \beta_2 x_2) = \beta_1 f(x_1) + \beta_2 f(x_2)$,
- If x_1 and x_2 are any two roots of f , then $f(x_1) = f(x_2) = f(x_1 + x_2) = 0$.

These two properties imply that all roots of a q -polynomial span an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$. Proposition 3.1 shows that the q -degree of a q -polynomial is lower bounded by the dimension of its root space. Definition 3.2 shows that given an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$, there exists a unique monic q -polynomial vanishing on such an $\mathbb{F}_q$ -subspace.

Proposition 3.1. *Let $f(x) \in \mathcal{L}[x]$ be a q -polynomial. Assume that $e_1, e_2, \dots, e_n \in \mathbb{F}_{q^m}$ span an $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$ of dimension r . If $f(e_i) = 0$ for all $i \in \{1..n\}$, then either $f(x) = 0$, or $\deg_q f(x) \geq r$.*

Definition 3.2 (Annihilator Polynomial [32]). *For any $\mathbb{F}_q$ -subspace E of $\mathbb{F}_{q^m}$ of dimension r , there exists a unique monic q -polynomial $\mathcal{A}_E(x)$ of q -degree r that vanishes on E , i.e., $\mathcal{A}_E(x) = 0$ for any $x \in E$. Such a monic q -polynomial is called the annihilator polynomial of E .*

Since the coordinates of any error of weight r uniquely span a r -dimensional $\mathbb{F}_q$ -subspace of $\mathbb{F}_{q^m}$, i.e., its r -dimensional support, we have Definition 3.3.

Definition 3.3 (Annihilator Polynomial of Vector). *For any $e = (e_1, e_2, \dots, e_n) \in \mathcal{S}_r^n$, let $\mathcal{A}_e(x)$ be the annihilator polynomial of the support of e . We call $\mathcal{A}_e(x)$ the annihilator polynomial of e . $\mathcal{A}_e(x)$ is unique, has q -degree r , and vanishes on e , i.e., $\mathcal{A}_e(e_i) = 0$ for all $i \in \{1..n\}$.*

Proposition 3.2 shows that the dimension of $\mathbb{F}_q$ -subspace spanned by some roots of a q -polynomial is upper bounded by its q -degree.

Proposition 3.2. *Let $f(x)$ be a q -polynomial of q -degree r . Let $\{e_i : f(e_i) = 0, i \in \{1..n\}\}$ be a set consisting of any n roots of $f(x)$. Then the dimension of the $\mathbb{F}_q$ -subspace spanned by such a set is at most r . In other words, if $\mathbf{e} = (e_1, e_2, \dots, e_n)$, then $\|\mathbf{e}\|_R \leq r$.*

Definition 3.4 (Gabidulin Codes [20]). *Let q, m, n, k be integers and $k \leq n \leq m$. Let $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ be the generator of weight n . A Gabidulin code of dimension k and length n generated by $\mathbf{g}$ is defined as*

$$\text{Gab}_k^{\mathbf{g}} = \left\{ f(\mathbf{g}) = (f(g_1), f(g_2), \dots, f(g_n)) : f(x) \in \mathcal{L}_{\leq k-1}[x] \right\}.$$

The generator matrix of $\text{Gab}_k^{\mathbf{g}}$ is

$$\mathbf{G} = \begin{bmatrix} g_1 & g_2 & \cdots & g_n \\ g_1^q & g_2^q & \cdots & g_n^q \\ \vdots & \vdots & \ddots & \vdots \\ g_1^{q^{k-1}} & g_2^{q^{k-1}} & \cdots & g_n^{q^{k-1}} \end{bmatrix}.$$

These codes benefit from efficient decoding algorithms correcting up to $\lfloor \frac{n-k}{2} \rfloor$ errors in a deterministic way.

Decoding Gabidulin Codes. There mainly are five ways to decode Gabidulin codes: Linearized Extended Euclidean Algorithm (LEEA) [20, 47], Berlekamp-Massey Like (BML) algorithm [36, 37], Gao-Like (GL) algorithm [48], Welch-Berlekamp Like (WBL) algorithm [8, 28], and Gaussian eliminations [8, 28]. The last two are essentially designed to solve the Linear Reconstruction (LR) problem arising from decoding AG codes. For our cryptographic parameters, the practical one is the WBL algorithm.

Given the Gabidulin codes in Definition 3.4, we assume that the message q -polynomial is $f(x) \in \mathcal{L}_{\leq k-1}[x]$, the error occurring at the channel is $\mathbf{e}$ and $\|\mathbf{e}\|_R \leq r$, the received word is $\mathbf{y} = f(\mathbf{g}) + \mathbf{e}$. The aim of decoding Gabidulin codes is to recover $\mathbf{e}$ and $f(x)$ from $\mathbf{y}$. The decoding Gabidulin codes problem is as follows.

Definition 3.5 (Decoding Gabidulin Code Problem DecGabCode($\mathbf{g}, \mathbf{y}$)).

Input : $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_R = n$; $\mathbf{y} = (y_1, y_2, \dots, y_n) \in \mathbb{F}_{q^m}^n$.

Output : $f(x) \in \mathcal{L}[x]$ and $\mathbf{e} \in \mathbb{F}_{q^m}^n$ such that: 1. $\deg_q f(x) \leq k-1$; 2. $\mathbf{e} \neq \mathbf{0}_n$ and $\|\mathbf{e}\|_R \leq r$; 3. $\mathbf{y} = f(\mathbf{g}) + \mathbf{e}$.

From Definition 3.3, we know that for any $\mathbf{e} \in \mathcal{S}_r^n$, there exists a unique annihilator polynomial $\mathcal{A}_{\mathbf{e}}(x)$ of q -degree r that vanishes on $\mathbf{e}$, i.e., $\mathcal{A}_{\mathbf{e}}(e_i) = 0$ for all $i \in \{1..n\}$. Thus, it is natural to transform the DecGabCode($\mathbf{g}, \mathbf{y}$) problem into the Non Linear Reconstruction (NLR) problem (Definition 3.6).

Definition 3.6 (Non Linear Reconstruction Problem NLR($\mathbf{g}, \mathbf{y}$)).

Input : $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_R = n$; $\mathbf{y} = (y_1, y_2, \dots, y_n) \in \mathbb{F}_{q^m}^n$.

Output : $f(x), v(x) \in \mathcal{L}[x]$ such that: 1. $\deg_q f(x) \leq k-1$; 2. $v(x) \neq 0$ and $\deg_q v(x) \leq r$; 3. $v(\mathbf{y}) = (v \circ f)(\mathbf{g})$.

Solving the NLR problem can be reduced to solving the LR problem in Definition 3.7.

Definition 3.7 (Linear Reconstruction Problem $\text{LR}(\mathbf{g}, \mathbf{y})$).

Input : $\mathbf{g} = (g_1, g_2, \dots, g_n) \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_{\mathbb{R}} = n$; $\mathbf{y} = (y_1, y_2, \dots, y_n) \in \mathbb{F}_{q^m}^n$.

Output : $v(x), u(x) \in \mathcal{L}[x]$ such that: 1. $v(x) \neq 0$ and $\deg_q v(x) \leq r$; 2. $\deg_q u(x) \leq k+r-1$; 3. $v(\mathbf{y}) = u(\mathbf{g})$.

In [8, 28], the $\text{NLR}(\mathbf{g}, \mathbf{y})$ and $\text{LR}(\mathbf{g}, \mathbf{y})$ problems are shown to be equivalent if $r \leq \lfloor \frac{n-k}{2} \rfloor$.

The WBL algorithm consists in solving the $\text{LR}(\mathbf{g}, \mathbf{y})$ problem, decomposed into two steps:

1. **Initialization step:** Two pairs of q -polynomials (u_0, v_0) and (u_1, v_1) satisfying

$$v_0(y_i) = u_0(g_i) \quad \text{and} \quad v_1(y_i) = u_1(g_i)$$

for $1 \leq i \leq k$ are computed. u_0 is set as the annihilator polynomial on $g_1, \dots, g_k$, u_1 is set as the q -polynomial interpolating $y_1, \dots, y_k$ on $g_1, \dots, g_k$, and

$$v_0(x) = 0 \quad \text{and} \quad v_1(x) = x.$$

2. **Interpolation step:** Iteratively, the q -degrees of (u_0, v_0) and (u_1, v_1) are increased using a recurrence relation ensuring that if the interpolation conditions

$$v(y_i) = u(g_i) \quad \text{for } 1 \leq i \leq j$$

are satisfied at step j , then they are also satisfied at step $j+1$. Besides, this construction ensures that at least one of the pairs satisfies the final degree conditions

$$\deg_q v(x) \leq r \quad \text{and} \quad \deg_q u(x) \leq k+r-1$$

when the q -polynomials are initialized according to the aforementioned Initialization step.

The correctness of the WBL algorithm is proved in [8]. Once the $\text{LR}(\mathbf{g}, \mathbf{y})$ problem is solved, one can solve $\text{DecGabCode}(\mathbf{g}, \mathbf{y})$ for Gabidulin codes as Algorithm 1:

Algorithm 1 Decoding errors of weight $\leq r$ by the WBL algorithm

Input: $q, m, n, k, r \in \mathbb{N}$, $\mathbf{g} \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{g}\|_{\mathbb{R}} = n$, $\mathbf{y} \in \mathbb{F}_{q^m}^n$.

Output: $f(x) \in \mathcal{L}_{\leq k-1}[x]$; $\mathbf{e} \in \mathbb{F}_{q^m}^n$ and $\|\mathbf{e}\|_{\mathbb{R}} \leq r$

- 1: Call the WBL algorithm from [8] (Algorithm 5)
 - 2: Return (N_1, W_1)
 - 3: Set $u(x) = N_1$ and $v(x) = W_1$
 - 4: Compute $f(x) = v(x) \setminus u(x)$
 - 5: **if** $\deg_q f(x) \leq k-1$ and $\|\mathbf{y} - f(\mathbf{g})\|_{\mathbb{R}} \leq r$:
 - 6: **return** $f(x)$ and $\mathbf{e} := \mathbf{y} - f(\mathbf{g})$
 - 7: **else:**
 - 8: **return** $\perp$
-

3.3 Public Key Encryption (BRQC.PKE)

Our BRQC uses three codes:

- A public $[n, k]_{q^m}$ -Gabidulin code with generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ generated by $\mathbf{g} \in \mathbb{F}_{q^m}^n$ of weight n . This Gabidulin code can correct up to weight $\lfloor \frac{n-k}{2} \rfloor$ by a deterministic decoding algorithm. We denote its decoding algorithm by Gab.Decode .

- A random $[2n, n]_{q^m}$ -ideal code with parity-check matrix $\begin{bmatrix} \mathbf{1} & \mathbf{h} \end{bmatrix}$.
- A random $[3n, n]_{q^m}$ -ideal code with a parity-check matrix $\begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{h} \\ \mathbf{0} & \mathbf{1} & \mathbf{s} \end{bmatrix}$.

The Gabidulin code is used in encryption and decryption steps. Two random ideal codes are used to ensure security. BRQC.PKE is described in Algorithms 2–5.

Correctness and security. In the decryption step, $\mathbf{v} - \mathbf{u} \cdot \mathbf{y} = \mathbf{m}\mathbf{G} + \mathbf{x} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y}$. The term $\mathbf{x} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y}$ is viewed as an error and $\|\mathbf{x} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y}\|_{\mathcal{R}} \leq w_{\mathbf{x}}w_{\mathbf{r}_2} + w_{\mathbf{y}}w_{\mathbf{r}_1} + w_{\mathbf{e}}$. Let $r = w_{\mathbf{x}}w_{\mathbf{r}_2} + w_{\mathbf{y}}w_{\mathbf{r}_1} + w_{\mathbf{e}}$. To obtain $\mathbf{m}$, one applies Gab.Decode to $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$. Thus, one must ensure that the weight $r \leq \lfloor \frac{n-k}{2} \rfloor$ and $\mathbf{m}$ can be decoded by Gab.Decode. Note that the decoding failure rate (DFR) is null. Then, the decryption of BRQC.PKE is deterministic, namely the decryption failure rate (also DFR) is null. In Section 6, we show that BRQC.PKE is tightly IND-CPA secure under the IBRS hardness assumption.

Algorithm 2 BRQC.PKE.Setup(λ)

Input: the security level parameter λ

Output: the global system parameters par

- 1: Set the security parameters $(q, m, n, k, w_{\mathbf{x}}, w_{\mathbf{y}}, w_{\mathbf{r}_1}, w_{\mathbf{r}_2}, w_{\mathbf{e}})$
 - 2: Sample $P(X) \in \mathbb{F}_q[X]$ (an irreducible polynomial of degree n) and build the polynomial ring $\mathcal{R} = \mathbb{F}_{q^m}[X]/\langle P(X) \rangle$
 - 3: Set the global system parameters $\text{par} = (q, m, n, k, w_{\mathbf{x}}, w_{\mathbf{y}}, w_{\mathbf{r}_1}, w_{\mathbf{r}_2}, w_{\mathbf{e}}, P(X))$
-

Algorithm 3 BRQC.PKE.KGen(par)

Input: the global system parameters par

Output: a public key $\text{pk} := (\mathbf{g}, \mathbf{h}, \mathbf{s})$ and a private key $\text{sk} := (\mathbf{x}, \mathbf{y})$

- 1: Sample $\mathbf{g} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^n$ from a seed θ_1
 - 2: Sample $\mathbf{h} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^n$ from a seed θ_1
 - 3: Sample $(\mathbf{x}, \mathbf{y}) \xleftarrow{\theta_2} \mathcal{S}_{(w_{\mathbf{x}}, w_{\mathbf{y}})}^{(n, n)}$ from a seed θ_2
 - 4: Compute $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$
-

Algorithm 4 BRQC.PKE.Enc($\text{pk}, \mathbf{m}; \theta$)

Input: the public key pk , a plaintext $\mathbf{m} \in \mathbb{F}_{q^m}^k$, and a seed θ

Output: a ciphertext $\text{ct} := (\mathbf{u}, \mathbf{v})$

- 1: Compute the generator matrix $\mathbf{G} \in \mathbb{F}_{q^m}^{k \times n}$ of $[n, k]_{q^m}$ -Gabidulin code defined by $\mathbf{g}$
 - 2: Sample $(\mathbf{r}_1, \mathbf{e}, \mathbf{r}_2) \xleftarrow{\theta} \mathcal{S}_{(w_{\mathbf{r}_1}, w_{\mathbf{e}}, w_{\mathbf{r}_2})}^{(n, n, n)}$
 - 3: Compute $\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$ and $\mathbf{v} = \mathbf{m}\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$
-

Algorithm 5 BRQC.PKE.Dec(sk, ct)

Input: the private key sk and the ciphertext ct

Output: the plaintext $\mathbf{m}$

- 1: Compute $\mathbf{v} - \mathbf{u} \cdot \mathbf{y} = \mathbf{m}\mathbf{G} + \mathbf{x} \cdot \mathbf{r}_2 + \mathbf{e} - \mathbf{r}_1 \cdot \mathbf{y}$
 - 2: Recover the plaintext $\mathbf{m} = \text{Gab.Decode}(\mathbf{v} - \mathbf{y} \cdot \mathbf{u})$
-

3.4 Key Encapsulation Mechanism (BRQC.KEM)

In this section, we consider KEM version of BRQC (BRQC.KEM), an IND-CCA2 secure KEM, from BRQC.PKE described in Algorithms 2–5 by the salted FO transformation [25]. The resulting BRQC.KEM is described in Algorithms 6–9.

Correctness and security. The correctness follows BRQC.PKE. BRQC.KEM has a null decapsulation failure rate. BRQC.KEM is IND-CCA2 secure. This security is tight with respect to the hardness of the IBRS assumption (see Section 6).

Algorithm 6 BRQC.KEM.Setup(λ)

Input: the security level parameter λ

Output: the global system parameters par

- 1: Set the global system parameters $\text{par} \leftarrow \text{BRQC.PKE.Setup}(\lambda)$
-

Algorithm 7 BRQC.KEM.KGen(par)

Input: the global system parameters par

Output: a public key pk and a private key sk

- 1: $(\text{pk}', \text{sk}') \leftarrow \text{BRQC.PKE.KGen}(\text{par})$
 - 2: Sample $\sigma \xleftarrow{\$} \mathbb{F}_{q^m}^k$
 - 3: Set the public key $\text{pk} := \text{pk}'$ and the private key $\text{sk} := (\text{sk}', \sigma)$
-

Algorithm 8 BRQC.KEM.Encaps(pk)

Input: the public key pk

Output: a ciphertext and key pair (ct, K)

- 1: Sample $\mathbf{m} \xleftarrow{\$} \mathbb{F}_{q^m}^k$ and salt $\xleftarrow{\$} \{0, 1\}^{\text{len}}$
 - 2: Compute $\theta = G(\text{pk}, \mathbf{m}, \text{salt})$
 - 3: $\text{ct}' \leftarrow \text{BRQC.PKE.Enc}(\text{pk}, \mathbf{m}; \theta)$
 - 4: $\text{ct} := (\text{ct}', \text{salt})$
 - 5: Compute $K = H(\text{pk}, \mathbf{m}, \text{ct}', \text{salt})$
-

Algorithm 9 BRQC.KEM.Decaps(sk, ct)

Input: the private key sk and the ciphertext ct

Output: the key K

- 1: Parse $\text{sk} := (\text{sk}', \sigma)$ and $\text{ct} := (\text{ct}', \text{salt})$
 - 2: $\overline{\mathbf{m}} \leftarrow \text{BRQC.PKE.Dec}(\text{sk}', \text{ct}')$ and compute $\overline{\theta} = G(\text{pk}, \overline{\mathbf{m}}, \text{salt})$
 - 3: $\overline{\text{ct}} \leftarrow \text{BRQC.PKE.Enc}(\text{pk}, \overline{\mathbf{m}}; \overline{\theta})$
 - 4: If $\overline{\text{ct}} \neq \text{ct}'$, set $K := H(\text{pk}, \sigma, \overline{\text{ct}}, \text{salt})$; else, set $K := H(\text{pk}, \overline{\mathbf{m}}, \overline{\text{ct}}, \text{salt})$
-

4 Parameters

In this section, we give concrete parameter sets. The proposed parameter sets cover three classical security levels: **128**-, **256**-, and **512**-bit security.

Each parameter set is chosen according to three principles.

- From Theorems 6.1 and 6.2, the security of schemes is tightly reduced to the hardness of two IBRS assumptions. Thus, the minimal complexity of known attacks against two assumptions at least reaches 2^λ for the classical security level parameter λ : 128, 256, and 512. The main attacks against the BRD problem are recalled in Section 7.
- The decoding capacity of the Gabidulin codes is ensured to satisfy the decryption correctness condition, namely decoding weight up to r .
- We set a sufficiently large plaintext space (i.e., $\mathbb{F}_{q^m}^k$). We assume that the number of queries to the hash oracles is q^{64} . In the classical model, by Theorem 6.2, we require that the size of the plaintext space (i.e., pts in Table 3, q^{mk}) at least satisfies: $|\mathbb{F}_{q^m}^k|/2^{64} \geq 2^\lambda$. Considering underlying quantum attacks, we set $\sqrt{|\mathbb{F}_{q^m}^k|/2^{64}} \geq 2^\eta$ where η is the required quantum security levels: 80, 128, and 256.

In Table 2, we select the security parameters for three security levels. In Table 3, we compute sizes, and estimate classical/quantum security strength and space complexity.

Table 2: Parameter sets

Instance	q	m	n	k	w_x	w_y	w_{r_1}	w_{r_2}	w_e	r	$P(X)$
BRQC-128	2	127	119	3	5	5	5	5	8	58	$X^{119} + X^8 + 1$
BRQC-256	2	163	161	3	5	6	6	7	8	79	$X^{161} + X^{18} + 1$
BRQC-512	2	229	227	3	6	7	8	8	8	112	$X^{227} + X^{10} + X^9 + X^4 + 1$

We set the weight t of generator g as m . $P(X)$ is a polynomial of degree n in $\mathbb{F}_q[X]$, and is used to build the ring $\mathcal{R} = \mathbb{F}_{q^m}[X]/\langle P(X) \rangle$ and construct the ideal codes (see Section 2.4). $P(X)$ is set to be an irreducible polynomial for resisting the folding attack [26]. Moreover, to decrease the computational costs, $P(X)$ is suggested to be sparse. We choose such $P(X)$ with the SageMath 9.5 software.

Table 3: Sizes, DFR, classical/quantum security strength, and space complexity

Instance	pts	sks	pks	cts	total	$ K $	DFR	Classical Security Strength	Quantum Security Strength	Space Complexity
BRQC-128	381	112	1954	3844	5798	64	Null	2^{181}	2^{90}	2^{181}
BRQC-256	489	126	3345	6626	9971	64	Null	2^{295}	2^{147}	2^{231}
BRQC-512	687	150	6562	13060	19622	64	Null	2^{566}	2^{283}	2^{278}

pts: the plaintext size, mk bits. **sks**: the private key size, $\lceil \frac{mk}{8} \rceil + 64$ bytes. **pks**: the public key size, $\lceil \frac{mn}{8} \rceil + 64$ bytes. **cts**: the ciphertext size, $2 \lceil \frac{mn}{8} \rceil + 64$ bytes. **total** = **pks** + **cts**. $|K|$: the shared key size. **DFR**: decapsulation failure rate. The classical security strength is described by the classical time complexity of the MM algebraic attack (see Table 6). The quantum security strength is the square root of the classical security strength. The space complexity corresponds to the classical security strength.

5 Performance

This section provides performance measures of our BRQC.KEM implementations.

Benchmark platform. All benchmarks were conducted inside a WSL 2 environment running Ubuntu 26.04 LTS on a Windows laptop equipped with an Intel® Core™ Ultra 9 275HX processor (8 performance cores, base frequency 2.70 GHz, with turbo frequencies up to 5.40 GHz,

16 efficient cores, 24 total threads) and 64 GB of memory. The software implementation environment is configured with support for the following instruction sets: MMX, SSE (including SSE, SSE2, SSE3, SSSE3, SSE4.1, SSE4.2), EM64T (Intel 64), AES, AVX, AVX2, AVX-VNNI, FMA3, and SHA. To avoid performance fluctuations caused by migration across the heterogeneous core types, each benchmark process was pinned to a single performance core (CPU 0, a P-core) via `sched_setaffinity`. The implementation was compiled with `gcc` version 13.3.0 using `-O3 -mavx2`, and relies on the RBC library (version 1.3) [3]. For each parameter set, the reported value is the mean over 50 random key instances; each optimized instance is timed over 100 inner iterations and each reference instance over 10 inner iterations. Since WSL 2 does not expose CPU frequency-scaling controls, Turbo Boost remains active and introduces rare slow samples that inflate the arithmetic mean of short-running operations. For the optimized implementation, we therefore report the median which is robust to such residual frequency jitter. The reference implementation runs long enough per sample that its distribution is unimodal, and its mean is reported directly.

Constant time. The provided implementations are conducted in a constant time way, so that their running time does not leak any information with respect to sensitive data such as the secret key, the weight of the error to be decoded by the Gabidulin code, or the validity of the received ciphertext. In particular, the Fujisaki–Okamoto re-encryption check in `Decaps` compares $(\mathbf{u}, \mathbf{v})$ and $(\mathbf{u}', \mathbf{v}')$ byte-by-byte with an OR-accumulator, and selects between the decoded message m and the implicit-rejection value σ using a constant-time bitmask instead of a branch.

5.1 Reference Implementation

The performance of our reference implementation on the aforementioned benchmark platform is described in Table 4. The reference implementation was compiled with `-std=c99 -Wpedantic -Wall -Wextra -O2`. Values are mean CPU cycles in kiloCycles.

Table 4: Performance of BRQC reference implementation (in kiloCycles)

Instance	KGen	Encaps	Decaps
BRQC-128	1 806.7	3 813.7	56 552.8
BRQC-256	8 287.4	17 531.4	258 716.6
BRQC-512	15 058.0	31 351.6	613 566.2

5.2 Optimized Implementation

An x86 performance-optimized implementation is provided. Its performances on the aforementioned benchmark platform are described in Table 5. The optimized implementation was compiled with `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -mpclmul -msse4.2 -maes -std=c99 -Wpedantic -Wall -Wextra -DSHAKE_TIMES4`. Values are median CPU cycles in kiloCycles.

6 Security Proof

In this section, we first prove that `BRQC.PKE` is a tight IND-CPA secure PKE under the IBRS hardness assumption, then obtain `BRQC.KEM` is a tight IND-CCA2 secure KEM under the same hardness assumption due to the salted FO transformation [25].

Table 5: Performance of BRQC optimized implementation (in kiloCycles)

Instance	KGen	Encaps	Decaps
BRQC-128	275.0	381.7	1 466.7
BRQC-256	1 183.7	1 357.7	8 352.4
BRQC-512	1 779.5	2 180.3	18 697.2

Theorem 6.1. *Under the $\text{DIBRSD}_{q,m,2n,n,w_{\mathbf{x}},w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}$ and $\text{DIBRSD}_{q,m,3n,n,w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}$ problems, our BRQC.PKE in Algorithms 2–5 is IND-CPA secure. Concretely, for any PPT algorithm $\mathcal{A}$, there exists a PPT algorithm $\mathcal{B}$ such that the advantage that $\mathcal{A}$ wins the game $\text{IND-CPA}_{\text{BRQC.PKE}}^{\mathcal{A}}(\lambda)$ defined by our BRQC.PKE is bounded as*

$$\text{Adv}_{\text{BRQC.PKE},\mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,2n,n,w_{\mathbf{x}},w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,3n,n,w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}}(\lambda).$$

Proof. Let $\text{PKE} = \text{BRQC.PKE}$. We prove the theorem using a sequence of games $\mathbf{G}_i$ for $i \in \{0, 1, 2\}$. We define

$$\text{Adv}_i := \Pr[\mathbf{G}_i \Rightarrow 1], \text{ for } i \in \{0, 1, 2\}.$$

Let $\mathbf{G}_0$ be the real IND-CPA game (see Figure 1) and define $\mathbf{G}_0 := \text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda)$. We introduce changes to $\mathbf{G}_0$, and end up at $\mathbf{G}_2$. We argue that $\mathbf{G}_2$ outputs 1 with a probability $1/2$. The games are presented in Figure 2.

Game $\mathbf{G}_0$: $\mathbf{G}_0$ is the real IND-CPA game and works as follows. First, it samples $\text{par} \leftarrow \text{Setup}(1^\lambda)$. Then, it computes the public key and the private key: sampling $\mathbf{g} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^n$ and $\mathbf{h} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^n$, sampling $(\mathbf{x}, \mathbf{y}) \xleftarrow{\theta_2} \mathcal{S}_{(w_{\mathbf{x}}, w_{\mathbf{y}})}^{(n,n)}$, and computing $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$. The public key $\text{pk} := (\mathbf{g}, \mathbf{h}, \mathbf{s})$ and the private key $\text{sk} := (\mathbf{x}, \mathbf{y})$. Next, $\mathcal{A}$ is given pk and par and samples two plaintexts $\mathbf{m}_0, \mathbf{m}_1$ with the equal length. The game samples $b \xleftarrow{\$} \{0, 1\}$, runs the encryption algorithm PKE.Enc , and computes the ciphertext $\text{ct} \leftarrow \text{PKE.Enc}(\text{pk}, \mathbf{m}_b, \theta)$ (see Line 08 in Figure 2). Finally, $\mathcal{A}$ outputs $b' \in \{0, 1\}$ with ct . The game outputs 1, if $b' = b$. By definition, we have

$$\text{Adv}_0 = \Pr[\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}(\lambda) \Rightarrow 1].$$

Game $\mathbf{G}_1$: We change $\mathbf{G}_0$ into $\mathbf{G}_1$. Recall that $\mathbf{s} = [\mathbf{1} \quad \mathbf{h}] \cdot \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}$. In this game, we sample $\mathbf{s} \xleftarrow{\$} \mathbb{F}_{q^m}^n$. We argue that two games are computationally close by the DIBRSD assumption. Thus, we have

$$|\text{Adv}_0 - \text{Adv}_1| \leq \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,2n,n,w_{\mathbf{x}},w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}}(\lambda).$$

Game $\mathbf{G}_2$: We change $\mathbf{G}_1$ into $\mathbf{G}_2$. Recall that $\begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{h} \\ \mathbf{0} & \mathbf{1} & \mathbf{s} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{e} \\ \mathbf{r}_2 \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{m}_b \mathbf{G} \end{bmatrix}$. In this

game, we sample $(\mathbf{u}, \mathbf{v}) \xleftarrow{\$} \mathbb{F}_{q^m}^n \times \mathbb{F}_{q^m}^n$ and replace $\begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{h} \\ \mathbf{0} & \mathbf{1} & \mathbf{s} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{e} \\ \mathbf{r}_2 \end{bmatrix}$ by $\begin{bmatrix} \mathbf{u} \\ \mathbf{v} - \mathbf{m}_b \mathbf{G} \end{bmatrix}$. We can

observe that $\mathbf{v} - \mathbf{m}_b \mathbf{G}$ is uniformly random due to the uniform randomness of $\mathbf{v}$. We argue that two games are computationally close by the DIBRSD assumption. We have

$$|\text{Adv}_1 - \text{Adv}_2| \leq \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,3n,n,w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}}(\lambda).$$

Game $\mathbf{G}_0\text{-}\mathbf{G}_2$	
01 $\text{par} \leftarrow \text{Setup}(1^\lambda)$	
02 $\mathbf{g} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^n, \mathbf{h} \xleftarrow{\theta_1} \mathbb{F}_{q^m}^n$	
03 $(\mathbf{x}, \mathbf{y}) \xleftarrow{\theta_2} \mathcal{S}_{(w_{\mathbf{x}}, w_{\mathbf{y}})}^{(n,n)}, \mathbf{s} = \begin{bmatrix} \mathbf{1} & \mathbf{h} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix}$	
04 $\mathbf{s} \xleftarrow{\$} \mathbb{F}_{q^m}^n$	// $\mathbf{G}_1$
05 $\text{pk} := (\mathbf{g}, \mathbf{h}, \mathbf{s}), \text{sk} := (\mathbf{x}, \mathbf{y})$	
06 $(\mathbf{m}_0, \mathbf{m}_1) \xleftarrow{\$} \mathbb{F}_{q^m}^k \times \mathbb{F}_{q^m}^k$	
07 $b \xleftarrow{\$} \{0, 1\}$	
08 $(\mathbf{r}_1, \mathbf{e}, \mathbf{r}_2) \xleftarrow{\theta} \mathcal{S}_{(w_{\mathbf{r}_1}, w_{\mathbf{e}}, w_{\mathbf{r}_2})}^{(n,n,n)}, \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{h} \\ \mathbf{0} & \mathbf{1} & \mathbf{s} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{r}_1 \\ \mathbf{e} \\ \mathbf{r}_2 \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{m}_b \mathbf{G} \end{bmatrix}$	
09 $(\mathbf{u}, \mathbf{v}) \xleftarrow{\$} \mathbb{F}_{q^m}^n \times \mathbb{F}_{q^m}^n, \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{u} \\ \mathbf{v} - \mathbf{m}_b \mathbf{G} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{m}_b \mathbf{G} \end{bmatrix}$	// $\mathbf{G}_2$
10 $\text{ct} = (\mathbf{u}, \mathbf{v})$	
11 $b' \leftarrow \mathcal{A}(\text{par}, \text{pk}, \text{ct})$	
12 $b' \neq b : \text{return } 0$	
13 return 1	

Figure 2: The games $\mathbf{G}_0\text{-}\mathbf{G}_2$ in the proof of Theorem 6.1. Annotated lines using “//” are only executed in the corresponding games.

At this point, in the game $\mathbf{G}_2$, all information $(\text{par}, \text{pk}, \text{ct})$ given to $\mathcal{A}$ is uniformly random and independent of the private key and the randomness. Thus,

$$\text{Adv}_2 = \Pr[\mathbf{G}_2 \Rightarrow 1] = \frac{1}{2}.$$

Combining equations obtained in games $\mathbf{G}_0 - \mathbf{G}_2$, we have

$$|\text{Adv}_0 - \text{Adv}_2| \leq \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,2n,n,w_{\mathbf{x}}+w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,3n,n,w_{\mathbf{r}_1}+w_{\mathbf{e}}+w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}}(\lambda).$$

$$\Longleftrightarrow$$

$$\left| \Pr[\text{IND-CPA}_{\text{PKE},\mathcal{A}}^A(\lambda) \Rightarrow 1] - \frac{1}{2} \right| \leq \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,2n,n,w_{\mathbf{x}}+w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,3n,n,w_{\mathbf{r}_1}+w_{\mathbf{e}}+w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}}(\lambda).$$

$$\Longleftrightarrow$$

$$\text{Adv}_{\text{PKE},\mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,2n,n,w_{\mathbf{x}}+w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}}(\lambda) + \text{Adv}_{\mathcal{B}}^{\text{DIBRSD}_{q,m,3n,n,w_{\mathbf{r}_1}+w_{\mathbf{e}}+w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}}(\lambda).$$

□

BRQC.KEM Security. BRQC.KEM is obtained by applying the salted FO transformation [25] to BRQC.PKE of a null DFR ($\delta = 0$). Following the salted FO transformation, an IND-CPA secure PKE tightly implies an IND-CCA2 secure KEM, our BRQC.KEM is an IND-CCA2 secure KEM. When the hash functions G and H are modeled as random oracles, by combining the generic bounds (Theorems 1 and 3 in [25]), we have Theorem 6.2.

Theorem 6.2. *Let $\mathcal{M}$ be the message space of BRQC.PKE. Assume that G and H are random oracles. For any classical adversary $\mathcal{A}'$ that makes at most q_G queries to G , q_H queries to H , and q_D queries to the decapsulation oracle, there exists an adversary $\mathcal{A}$ of roughly the same running time as that of $\mathcal{A}'$ such that*

$$\text{Adv}_{\text{BRQC.KEM}, \mathcal{A}'}^{\text{IND-CCA2}}(\lambda) \leq 2\text{Adv}_{\text{BRQC.PKE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) + \frac{1}{|\mathcal{M}| \cdot 2^{\text{len}}} + \frac{2q_H + q_G}{|\mathcal{M}|}.$$

Combining Theorems 6.1 and 6.2, we have that the IND-CCA2 security of BRQC.KEM is tight with respect to the hardness of the $\text{DIBRSD}_{q,m,2n,n,w_x+w_y}^{2,(n,n),(w_x,w_y)}$ and $\text{DIBRSD}_{q,m,3n,n,w_{r_1}+w_e+w_{r_2}}^{3,(n,n,n),(w_{r_1},w_e,w_{r_2})}$ assumptions.

7 Known Attacks on the BRSD Problem

The BRSD problem and the BRD problem are equivalent. Currently, the powerful combinatorial and algebraic attacks start with the BRD problem. From the BRD problem, launching these attacks needs the following Theorem 7.1.

Theorem 7.1. *Solving the BRD problem defined by an $[n, k]_{q^m}$ linear code $\mathcal{C}$ and instantiated by (G, y, e) is equivalent to finding a codeword belonging to $\mathcal{S}_r^n$ in the $[n, k+1]_{q^m}$ extended code $\mathcal{C}_y$ of $\mathcal{C}$.*

Proof. Once obtaining word y , one adds y to $\mathcal{C}$ and obtains an $[n, k+1]_{q^m}$ extended code $\mathcal{C}_y = \mathcal{C} + \langle y \rangle$ with a generator matrix $\begin{bmatrix} y \\ G \end{bmatrix}$. In this way, $e = (1 - m) \begin{bmatrix} y \\ G \end{bmatrix}$ is exactly a codeword of $\mathcal{C}_y$. Let $G_y \in \mathbb{F}_{q^m}^{(k+1) \times n}$ be a generator matrix of $\mathcal{C}_y$ and $H_y \in \mathbb{F}_{q^m}^{(n-k-1) \times n}$ be a parity-check matrix of $\mathcal{C}_y$. Then solving the BRD problem is equivalent to finding an $u \in \mathbb{F}_{q^m}^{k+1}$ such that

$$uG_y = e, \tag{1}$$

or finding an error $e \in \mathcal{S}_r^n$ such that

$$eH_y^\top = 0_{n-k-1}. \tag{2}$$

□

The Structure of Standard Errors. Recalled that any random standard error $e \in \mathcal{S}_r^n$ can be expressed as $e = \varepsilon C = \alpha S C$ where $\varepsilon \in \mathbb{F}_{q^m}^r$ is e 's basis, $S \in \mathbb{F}_q^{m \times r}$ is e 's support matrix, and $C \in \mathbb{F}_q^{r \times n}$ is e 's coefficient matrix. S (or C) can be reduced to a systematic form, namely the first r rows (or the first r columns) form an identity matrix I_r .

The Structure of Blockwise Errors. For any random blockwise error $e = (e_1, e_2, \dots, e_\ell) \in \mathcal{S}_r^n$, there are a basis ε and a coefficient matrix C to determine e :

$$e = \varepsilon C = \alpha S C \tag{3}$$

$$\varepsilon = (\varepsilon_1, \varepsilon_2, \dots, \varepsilon_\ell) \in \mathbb{F}_{q^m}^r, \quad S = (S_1, S_2, \dots, S_\ell) \in \mathbb{F}_q^{m \times r}, \quad C = \text{Diag}(C_1, C_2, \dots, C_\ell) \in \mathbb{F}_q^{r \times n},$$

where $\mathbf{e}_i \in \mathbb{F}_{q^m}^{r_i}$ is $\mathbf{e}_i$'s basis, $\mathbf{S}_i \in \mathbb{F}_q^{m \times r_i}$ is $\mathbf{e}_i$'s support matrix, and $\mathbf{C}_i \in \mathbb{F}_q^{r_i \times n_i}$ is $\mathbf{e}_i$'s coefficient matrix. $\mathbf{S}_i$ (or $\mathbf{C}_i$) can be reduced as systematic form, namely the first r_i rows (or the first r_i columns) form an identity matrix $\mathbf{I}_{r_i}$.

7.1 Combinatorial Attacks

The typical combinatorial attacks on the BRD problem have been considered in [5, 40, 41]. The powerful attacks are the AGHT attack, the PRR attack, and the OJ attack.

The AGHT Attack. The AGHT attack [7] is proposed by Aragon, Gaborit, Hauteville, and Tillich for solving the RD problem and is an improvement of GRS combinatorial attack [21]. When the AGHT attack is adapted into the BRD problem, the solver views the blockwise error of weight $\mathbf{r}$ as a standard error of weight r and performs the AGHT attack. The attack is divided into two strategies. Here we call **AGHT-S** and **AGHT-C**, respectively. They guess a subspace that respectively contains the column space of $\mathbf{S}$ and the row space of $\mathbf{C}$, and check if the guess is correct by solving the linear system obtained from the parity-check equations. The former is the case of cryptographic parameters in this submission.

AGHT-S: The solver guesses a subspace that contains the column space of $\mathbf{S}$.

- Let E be the r -dimensional support of $\mathbf{e}$. It guesses a t -dimensional $\mathbb{F}_q$ -subspace F such that F contains E . Equivalently, it guesses randomly a t -dimensional subspace T of $\mathbb{F}_q^m$ such that T contains the r -dimensional column space of $\mathbf{S}$. Let $\mathbf{T} \in \mathbb{F}_q^{m \times t}$ be the basis matrix of T . Let $\mathbf{f} = \alpha \mathbf{T} \in \mathbb{F}_{q^m}^t$. In this way, the $\mathbb{F}_q$ -subspace spanned by $\mathbf{f}$ is t -dimensional and contains E . We denote such a t -dimensional $\mathbb{F}_q$ -subspace as F , whose basis is $\mathbf{f}$. Due to $F \supset E$, $\mathbf{e}$ can be expressed by the basis $\mathbf{f}$ of F . Then there exists a matrix $\mathbf{F} \in \mathbb{F}_q^{t \times n}$ such that $\mathbf{e} = \mathbf{fF}$.
- Express the blockwise error $\mathbf{e}$ as $\mathbf{e} = \mathbf{fF} \in \mathbb{F}_{q^m}^n$.
- From the parity-check Equation (2): $\mathbf{eH}_y^\top = \mathbf{0}$, construct a linear system $\mathbf{fFH}_y^\top = \mathbf{0}$ and unfold it over $\mathbb{F}_q$ to obtain a linear system with $\sum_{i=1}^\ell n_i t_i$ unknowns in the entries of $\mathbf{F}$ and $m(n-k-1)$ equations over $\mathbb{F}_q$. (see [41] for the details of the unfolding operation.)
- Solve the obtained linear system for a single solution as long as $nt \leq m(n-k-1)$. (If the system could be solved, then the guess is correct.)
- Compute $\mathbf{fF}$ as $\mathbf{e}$ if the entries of $\mathbf{F}$ are solved.

The most costly part of the attack consists in successfully finding F containing E . This probability is estimated as $q^{r(m-t)}$.

Then one uses $\mathbb{F}_{q^m}$ -linearity to improve the success probability and reduce the cost. Since, for almost any $\beta \in \mathbb{F}_{q^m}^*$, $\|\beta \mathbf{e}\|_R = r$, $\text{Supp}(\beta \mathbf{e}) = \beta \text{Supp}(\mathbf{e}) = \beta E$, and all multiples $\beta \mathbf{e}$ are solutions of Equation (2): $\mathbf{eH}_y^\top = \mathbf{0}$, one can find the solution of form $\beta \mathbf{e}$. Then one can guess T such that the t -dimensional $\mathbb{F}_q$ -subspace F spanned by $\mathbf{f}$ contains the r -dimensional support of the form βE . For the fixed E , the number of βE is at most $\frac{q^m-1}{q-1}$. The probability of $F \supset \beta E$ is estimated as $\frac{q^m-1}{q-1} q^{-r(m-t)} \approx q^{-r(m-t)+m}$. As a result, this strategy has a complexity of $\mathcal{O}((m(n-k-1))^\omega q^{r(m-t)-m})$ over $\mathbb{F}_q$.

AGHT-C: The solver guesses a subspace that contains the row space of $\mathbf{C}$.

- Guess randomly t -dimensional subspace F' of $\mathbb{F}_q^n$ such that F' contains the r -dimensional row space C of $\mathbf{C}$. Let the matrix $\mathbf{F}' \in \mathbb{F}_q^{t' \times n}$ be a basis matrix of F' . Then there exists $\mathbf{f}' \in \mathbb{F}_{q^m}^{t'}$ such that $\mathbf{e} = \mathbf{f}' \mathbf{F}'$.
- Express the blockwise error $\mathbf{e}$ as $\mathbf{e} = \mathbf{f}' \mathbf{F}' \in \mathbb{F}_{q^m}^n$.
- From the parity-check equation: $\mathbf{e} \mathbf{H}^\top = \mathbf{s}$, construct a linear system $\mathbf{f}' \mathbf{F}' \mathbf{H}^\top = \mathbf{s}$ with t' unknowns in the entries of $\mathbf{f}'$ and $n - k$ equations over $\mathbb{F}_{q^m}$.
- Solve the obtained linear system for single solution $\mathbf{f}'$ as long as $t' \leq n - k$. (If the system could be solved, then the guess is correct.)
- Compute $\mathbf{f}' \mathbf{F}'$ as $\mathbf{e}$ if $\mathbf{f}'$ is solved.

The most costly part of the attack consists in successfully finding F' containing C . This probability is estimated as $q^{-r(n-t')}$. The fourth step costs $\mathcal{O}((n-k)\omega m^2)$ operations over $\mathbb{F}_q$. As a result, this strategy has a complexity of $\mathcal{O}((n-k)\omega m^2 q^{rk})$ over $\mathbb{F}_q$.

Theorem 7.2. *The time complexity of solving the BRD problem by the AGHT attack is estimated as*

$$\min \left(((n-k-1)m)^\omega q^{r \lceil \frac{(k+1)m}{n} \rceil - m}, (n-k)\omega m^2 q^{rk} \right)$$

over $\mathbb{F}_q$ where ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$.

The PRR Attack. The PRR attack is proposed by Puchinger, Renner, and Rosenkilde in [35], where the authors adapted the GRS attack [21] to the sum-rank decoding problem with equal block length. In [41], the authors generalized the block length, adapted the ideas of the AGHT attack [7] to the BRD problem, and obtained the gain of m bits by $\mathbb{F}_{q^m}$ -linearity. The attack is divided into two strategies. Here we call **PRR-S** and **PRR-C**, respectively. They guess a subspace that contains the column space of all $\mathbf{S}_i$ or the row space of all $\mathbf{C}_i$, and check if the guess is correct by solving the linear system obtained from the parity-check equations.

PRR-S: The solver guesses subspaces that contain the column space of $\mathbf{S}_i$.

- For $i \in \{1..\ell\}$, let the E_i be r_i -dimensional support of $\mathbf{e}_i$. It guesses a t_i -dimensional $\mathbb{F}_q$ -subspace F_i such that F_i contains E_i . Equivalently, it guesses randomly a t_i -dimensional subspace T_i of $\mathbb{F}_q^m$ such that T_i contains the r_i -dimensional column space of $\mathbf{S}_i$. Let $\mathbf{T}_i \in \mathbb{F}_q^{m \times t_i}$ be the basis matrix of T_i . Let $\mathbf{f}_i = \alpha \mathbf{T}_i \in \mathbb{F}_{q^m}^{t_i}$. In this way, the $\mathbb{F}_q$ -subspace spanned by $\mathbf{f}_i$ is t_i -dimensional and contains E_i . We denote such a t_i -dimensional $\mathbb{F}_q$ -subspace as F_i , whose basis is $\mathbf{f}_i$. Due to $F_i \supset E_i$, $\mathbf{e}_i$ can be expressed by the basis $\mathbf{f}_i$ of F_i . Then there exists a matrix $\mathbf{F}_i \in \mathbb{F}_q^{t_i \times n_i}$ such that $\mathbf{e}_i = \mathbf{f}_i \mathbf{F}_i$. At this point, $\sum_{i=1}^\ell F_i \supset \sum_{i=1}^\ell E_i$ where $\sum_{i=1}^\ell E_i$ is the support of $\mathbf{e}$, and a basis of $\sum_{i=1}^\ell F_i$ is $(\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_\ell)$.
- Express the blockwise error $\mathbf{e}$ as

$$\mathbf{e} = \underbrace{(\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_\ell)}_{:= \mathbf{f} \in \mathbb{F}_{q^m}^{\sum_{i=1}^\ell t_i}} \underbrace{\text{Diag}(\mathbf{F}_1, \mathbf{F}_2, \dots, \mathbf{F}_\ell)}_{:= \mathbf{F} \in \mathbb{F}_q^{\sum_{i=1}^\ell t_i \times n}} := \mathbf{f} \mathbf{F} \in \mathbb{F}_{q^m}^n.$$

- From the parity-check Equation (2): $\mathbf{e} \mathbf{H}_y^\top = \mathbf{0}$, construct a linear system $\mathbf{f} \mathbf{F} \mathbf{H}_y^\top = \mathbf{0}$ and unfold it over $\mathbb{F}_q$ to obtain a linear system with $\sum_{i=1}^\ell n_i t_i$ unknowns in the entries of $\mathbf{F}$ and $m(n-k-1)$ equations over $\mathbb{F}_q$. (The unfolding operation is similar to the third step of the AGHT-S attack.)

- Solve the obtained linear system for a single solution as long as $\sum_{i=1}^{\ell} n_i t_i \leq m(n-k-1)$. (If the system could be solved, then the guess is correct.)
- Compute $\mathbf{f}\mathbf{F}$ as $\mathbf{e}$ if the entries of $\mathbf{F}$ are solved.

The most costly part of the attack consists in successfully finding F_i containing E_i for $i \in \{1..\ell\}$. This probability is estimated as $q^{-\sum_{i=1}^{\ell} r_i(m-t_i)}$ because the probability of $F_i \supset E_i$ is $q^{-r_i(m-t_i)}$.

Then one uses $\mathbb{F}_{q^m}$ -linearity to improve the success probability and reduce the cost. Since, for almost any $\beta \in \mathbb{F}_{q^m}^*$, $\|\beta\mathbf{e}\|_R = r$, $\text{Supp}(\beta\mathbf{e}) = \beta\text{Supp}(\mathbf{e}) = \beta\sum_{i=1}^{\ell} E_i = \sum_{i=1}^{\ell} \beta E_i$, and all multiples $\beta\mathbf{e}$ are solutions of Equation (2): $\mathbf{e}\mathbf{H}_y^\top = \mathbf{0}$, one can find the solution of form $\beta\mathbf{e}$. Then one can guess T_i such that the t_i -dimensional $\mathbb{F}_q$ -subspace F_i spanned by $\mathbf{f}_i$ contains the r_i -dimensional support of the form βE_i . In this way, $\sum_{i=1}^{\ell} F_i \supset \sum_{i=1}^{\ell} \beta E_i = \beta\sum_{i=1}^{\ell} E_i$. For the fixed E_i 's, the number of $\beta\sum_{i=1}^{\ell} E_i$ is at most $\frac{q^m-1}{q-1}$. The probability of $\sum_{i=1}^{\ell} F_i \supset \sum_{i=1}^{\ell} \beta E_i$ is estimated as $\frac{q^m-1}{q-1} q^{-\sum_{i=1}^{\ell} r_i(m-t_i)} \approx q^{-\sum_{i=1}^{\ell} r_i(m-t_i)+m}$. As a result, this strategy has a complexity of $\mathcal{O}\left((m(n-k-1))^\omega q^{\sum_{i=1}^{\ell} r_i(m-t_i)-m}\right)$ over $\mathbb{F}_q$.

PRR-C: The solver guesses subspaces that contain the row space of C_i .

- For $i \in \{1..\ell\}$, guess randomly t'_i -dimensional subspace F'_i of $\mathbb{F}_q^{n_i}$ such that F'_i contains the r_i -dimensional row space C_i of C_i . Let the matrix $\mathbf{F}'_i \in \mathbb{F}_q^{t'_i \times n_i}$ be a basis matrix of F'_i . Then there exists $\mathbf{f}'_i \in \mathbb{F}_{q^m}^{t'_i}$ such that $\mathbf{e}_i = \mathbf{f}'_i \mathbf{F}'_i$.
- Express the blockwise error $\mathbf{e}$ as

$$\mathbf{e} = \underbrace{(\mathbf{f}'_1, \mathbf{f}'_2, \dots, \mathbf{f}'_\ell)}_{:= \mathbf{f}' \in \mathbb{F}_{q^m}^{\sum_{i=1}^{\ell} t'_i}} \underbrace{\text{Diag}(\mathbf{F}'_1, \mathbf{F}'_2, \dots, \mathbf{F}'_\ell)}_{:= \mathbf{F}' \in \mathbb{F}_q^{\sum_{i=1}^{\ell} t'_i \times n}} := \mathbf{f}' \mathbf{F}' \in \mathbb{F}_{q^m}^n.$$

- From the parity-check equation: $\mathbf{e}\mathbf{H}^\top = \mathbf{s}$, construct a linear system $\mathbf{f}' \mathbf{F}' \mathbf{H}_y^\top = \mathbf{0}$ with $\sum_{i=1}^{\ell} t'_i$ unknowns in the entries of $\mathbf{f}'$ and $n-k$ equations over $\mathbb{F}_{q^m}$.
- Solve the obtained linear system for single solution $\mathbf{f}'$ as long as $\sum_{i=1}^{\ell} t'_i \leq n-k$. (If the system could be solved, then the guess is correct.)
- Compute $\mathbf{f}' \mathbf{F}'$ as $\mathbf{e}$ if $\mathbf{f}'$ is solved.

The most costly part of the attack consists in successfully finding F'_i containing C_i for $i \in \{1..\ell\}$. This probability is estimated as $q^{-\sum_{i=1}^{\ell} r_i(n_i-t'_i)}$ because the probability of $C_i \subset F'_i$ is $q^{-r_i(n_i-t'_i)}$. The fourth step costs $\mathcal{O}\left((n-k)^\omega m^2\right)$ operations over $\mathbb{F}_q$. As a result, this strategy has a complexity of $\mathcal{O}\left((n-k)^\omega m^2 q^{\sum_{i=1}^{\ell} r_i(n_i-t'_i)}\right)$ over $\mathbb{F}_q$.

Theorem 7.3. *The time complexity of solving the BRD problem by the PRR attack is estimated as*

$$\min \left((m(n-k-1))^\omega q^{\sum_{i=1}^{\ell} r_i(m-t_i)-m}, (n-k)^\omega m^2 q^{\sum_{i=1}^{\ell} r_i(n_i-t'_i)} \right)$$

over $\mathbb{F}_q$ where ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$, t_i is chosen to minimize $\sum_{i=1}^{\ell} r_i(m-t_i) - m$ under the constraints

$$\begin{cases} r_i \leq t_i \leq m, & \text{for } i \in \{1..\ell\}; \\ \sum_{i=1}^{\ell} t_i \leq m; \\ \sum_{i=1}^{\ell} n_i t_i \leq m(n-k-1), \end{cases}$$

t'_i is chosen to minimize $\sum_{i=1}^{\ell} r_i(n_i - t'_i)$ under the constraints

$$\begin{cases} r_i \leq t'_i \leq n_i, & \text{for } i \in \{1..\ell\}; \\ \sum_{i=1}^{\ell} t'_i \leq n - k. \end{cases}$$

The OJ Attack. The OJ attack [33] is proposed by Ourivski and Johansson for the RD problem. In [40, 41], the OJ attack is adapted to the BRD problem. Recently, this adaptation is proven to be overly optimistic, and a practical OJ attack is refined in [38]. The general idea of the OJ attack is that the solver constructs a quadratic multivariate system about the entries of matrices $\mathbf{S}$ and $\mathbf{C}$ from Equation (1): $\mathbf{uG}_y = \mathbf{e}$, then guesses the entries of $\mathbf{S}$ (resp. $\mathbf{C}$) to transform the quadratic multivariate system into a linear system about the entries of $\mathbf{C}$ (resp. $\mathbf{S}$). The cost depends on the probability of successfully guessing the entries of the matrices.

We start with Theorem 7.1. Let $\mathbf{G}_y = [\mathbf{I}_{k+1} \ \mathbf{R}] \in \mathbb{F}_{q^m}^{(k+1) \times n}$ be a systematic generator matrix of $\mathcal{C}_y$ where $\mathbf{R} \in \mathbb{F}_{q^m}^{(k+1) \times (n-k-1)}$. Let $\bar{\mathbf{e}}_1$ and $\bar{\mathbf{e}}_2$ be the first $k+1$ and the last $n-k-1$ coordinates of $\mathbf{e}$, respectively. Let $\mathbf{A}_1$ and $\mathbf{A}_2$ be the first $k+1$ columns and the last $n-k-1$ columns of $\mathbf{C}$, respectively. Then

$$\begin{aligned} \mathbf{e} = (\bar{\mathbf{e}}_1, \bar{\mathbf{e}}_2) &= \alpha \mathbf{S} \mathbf{C} = \alpha \mathbf{S} [\mathbf{A}_1, \mathbf{A}_2] = (\alpha \mathbf{S} \mathbf{A}_1, \alpha \mathbf{S} \mathbf{A}_2) \\ &\iff \\ \bar{\mathbf{e}}_1 &= \alpha \mathbf{S} \mathbf{A}_1, \quad \bar{\mathbf{e}}_2 = \alpha \mathbf{S} \mathbf{A}_2. \end{aligned} \tag{4}$$

From Equation (1), we have

$$\mathbf{uG}_y = \mathbf{e} \iff (\mathbf{u} \ \mathbf{uR}) = (\bar{\mathbf{e}}_1, \bar{\mathbf{e}}_2) \iff \bar{\mathbf{e}}_1 \mathbf{R} = \bar{\mathbf{e}}_2. \tag{5}$$

Combining Equation (4) and Equation (5), we have

$$\alpha \mathbf{S} \mathbf{A}_1 \mathbf{R} = \alpha \mathbf{S} \mathbf{A}_2. \tag{6}$$

By Equation (6), for $j \in \{1..n-k-1\}$, let $\mathbf{r}_j$ and $\mathbf{a}_j$ be the j -th column of $\mathbf{R}$ and $\mathbf{A}_2$, respectively, then

$$\alpha \mathbf{S} \mathbf{A}_1 \mathbf{r}_j = \alpha \mathbf{S} \mathbf{a}_j \iff \alpha \mathbf{S} [\mathbf{A}_1 \ \mathbf{a}_j] \begin{bmatrix} \mathbf{r}_j \\ -1 \end{bmatrix} = 0. \tag{7}$$

Let J be a fixed and random subset of $\{1..n-k-1\}$ and $\#J = N$. The OJ modeling is:

$$\left\{ P_j = \alpha \mathbf{S} [\mathbf{A}_1 \ \mathbf{a}_j] \begin{bmatrix} \mathbf{r}_j \\ -1 \end{bmatrix} : j \in J \right\} \tag{8}$$

Unknowns: $mr + (k+1+N)r$ entries of $\mathbf{S}$, $\mathbf{A}_1$, and $\mathbf{a}_j$ over $\mathbb{F}_q$,

Equations: N quadratic multivariate equations $P_j = 0$ over $\mathbb{F}_{q^m}$.

These equations are define over $\mathbb{F}_{q^m}$, but the unknowns are in the basis field $\mathbb{F}_q$. Since $mr + (k+1+N)r \gg N$, the modeling is underdetermined and has many solutions. However, one wants more equations such that the modeling is overdetermined for a unique solution. To obtain more equations, one unfolds the coefficients of P_j over $\mathbb{F}_q$. The resulting modeling includes:

Unknowns: $mr + (k+1+N)r$ entries of $\mathbf{S}$, $\mathbf{A}_1$, and $\mathbf{a}_j$ over $\mathbb{F}_q$,

Equations: mN quadratic multivariate equations over $\mathbb{F}_q$.

Once the OJ modeling is solved, one can obtain the entries of $\mathbf{S}$. Then from Equation (2): $\mathbf{e} \mathbf{H}_y^\top = \alpha \mathbf{S} \mathbf{C} \mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$, one can obtain $\mathbf{C}$. Finally, one determines $\mathbf{e}$ by computing $\alpha \mathbf{S} \mathbf{C}$.

The idea of solving the OJ modeling is that the solver transforms the quadratic multivariate system into a linear system with mN equations. The transformation includes the basis enumeration and the coordinate enumeration. The former guesses the entries of $\mathbf{S}$ to solve a linear system in the entries of $[\mathbf{A}_1 \ \mathbf{a}_j]$ to obtain $\mathbf{S}$. The latter guesses the entries of $[\mathbf{A}_1 \ \mathbf{a}_j]$ to solve a linear system in the entries of $\mathbf{S}$ to obtain $\mathbf{S}$. Thus, we here call **OJ-S** and **OJ-C**, respectively. For the coordinate enumeration, to obtain the whole $\mathbf{S}$, we must carefully choose $\mathbf{a}_j$ to construct a linear system whose unknowns include *all* entries of $\mathbf{S}$. The guess cost dominates the complexity of the OJ attack. A smaller number of entries in $\mathbf{S}$ (resp. $[\mathbf{A}_1 \ \mathbf{a}_j]$) leads to a more powerful basis enumeration (resp. coordinate enumeration). To obtain the best complexity, we must reduce the support matrix $\mathbf{S}$ and the coefficient matrix $\mathbf{C}$.

Reduced Support/Coefficient Matrices for Blockwise Errors We here recall that the reduction of the blockwise errors in [40, 41]. The reduction uses two assumptions:

- The first coordinate of $\mathbf{e}$ be 1. Then one can set the first column of $\mathbf{C}$ to $(1 \ 0 \ \dots \ 0)^\top$ and the first column of $\mathbf{S}$ to $(1 \ 0 \ \dots \ 0)^\top$.
- The first r_i coordinates of $\mathbf{e}_i$ are independent over $\mathbb{F}_q$.

With these two assumptions, $\mathbf{S}$ and $\mathbf{C}$ can be further reduced to two forms with fewer entries.

- $\mathbf{S}_{\{1..r\},*} = \mathbf{I}_r$. By Gaussian elimination on columns of $\mathbf{S}$, there exists a matrix $\mathbf{P} \in \text{GL}_r(\mathbb{F}_q)$ such that $\mathbf{S}\mathbf{P} = \left[\begin{array}{c|c} \mathbf{I}_r & \mathbf{S}' \end{array} \right]$ and $\mathbf{P}^{-1}\mathbf{C} = \left[\begin{array}{c|c} 1 & \mathbf{C}' \end{array} \right]$ where $\mathbf{S}' \in \mathbb{F}_q^{(m-r) \times (r-1)}$ and $\mathbf{C}' \in \mathbb{F}_q^{r \times (n-1)}$. Then

$$\mathbf{e} = \alpha \mathbf{S} \mathbf{C} = \alpha \mathbf{S} \mathbf{P} \mathbf{P}^{-1} \mathbf{C} = \alpha \left[\begin{array}{c|c} \mathbf{I}_r & \mathbf{S}' \end{array} \right] \left[\begin{array}{c|c} 1 & \mathbf{C}' \end{array} \right]. \quad (9)$$

The reduced support matrix is $\mathbf{S} := \mathbf{S}\mathbf{P}$ and the reduced coefficient matrix is $\mathbf{C} := \mathbf{P}^{-1}\mathbf{C}$.

- $\mathbf{C}_i$ is systematic form. By Gaussian elimination on row of $\mathbf{C}$, there exists a matrix $\mathbf{Q} \in \text{GL}_r(\mathbb{F}_q)$ such that $\mathbf{S}\mathbf{Q}^{-1} = \left[\begin{array}{c|c} 1 & \mathbf{S}' \end{array} \right]$ and $\mathbf{Q}_i \mathbf{C}_i = [\mathbf{I}_{r_i} \ \mathbf{C}'_i]$ where $\mathbf{S}' \in \mathbb{F}_q^{m \times (r-1)}$ and $\mathbf{C}'_i \in \mathbb{F}_q^{r_i \times (n_i - r_i)}$. Then

$$\mathbf{e} = \alpha \mathbf{S} \mathbf{Q}^{-1} \mathbf{Q} \mathbf{C} = \alpha \left[\begin{array}{c|c} 1 & \mathbf{S}' \end{array} \right] \begin{bmatrix} [\mathbf{I}_{r_1} \ \mathbf{C}'_1] & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & [\mathbf{I}_{r_2} \ \mathbf{C}'_2] & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & [\mathbf{I}_{r_\ell} \ \mathbf{C}'_\ell] \end{bmatrix}. \quad (10)$$

The reduced support matrix is $\mathbf{S} := \mathbf{S}\mathbf{Q}^{-1}$ and the reduced coefficient matrix is $\mathbf{C} := \mathbf{Q}\mathbf{C}$.

Theorem 7.4. For general ℓ , when $\mathbf{S}$ and $\mathbf{C}$ are reduced as in Equation (9), the OJ attack with the basis enumeration (OJ-S) solves the BRD problems with complexity $\mathcal{O}((mN)^\omega q^{(m-r)(r-1)})$ over $\mathbb{F}_q$ where $N = \left\lceil \frac{kr}{m-r} \right\rceil$ and $N \leq n - k - 1$.

Theorem 7.5. For general ℓ , when $\mathbf{S}$ and $\mathbf{C}$ are reduced as in Equation (10), the OJ attack with the coordinate enumeration (OJ-C) solves the BRD problem with complexity over $\mathbb{F}_q$

$$\left\{ \begin{array}{ll} \mathcal{O} \left((mN)^\omega q^{(N - \sum_{i=1}^\ell r_i + k + 1)\gamma} \right), & \text{if } 0 < k + 1 \leq r_1; \\ \mathcal{O} \left((mN)^\omega q^{(r_1 - 1)(k + 1 - r_1) + (N - \sum_{i=2}^\ell r_i)\gamma} \right), & \text{if } r_1 < k + 1 \leq n_1; \\ \mathcal{O} \left((mN)^\omega q^{\sum_{i=1}^j r_i(n_i - r_i) - (n_1 - r_1) + (N - \sum_{i=1}^j n_i - \sum_{i=j+1}^\ell r_i + k + 1)\gamma} \right), & \text{if } \sum_{i=1}^j n_i < k + 1 \leq \sum_{i=1}^j n_i + r_{j+1}; \\ \mathcal{O} \left((mN)^\omega q^{\sum_{i=1}^j r_i(n_i - r_i) - (n_1 - r_1) + (k + 1 - \sum_{i=1}^j n_i - r_{j+1})r_{j+1} + (N - \sum_{i=j+2}^\ell r_i)\gamma} \right), & \text{if } \sum_{i=1}^j n_i + r_{j+1} < k + 1 \leq \sum_{i=1}^{j+1} n_i, \end{array} \right.$$

where $\gamma = \min \{r_i : i \in \{1..\ell\}\}$, $j \in \{1..\ell - 1\}$, and $N = \left\lceil \frac{m(r-1) + n_1 - r_1}{m} \right\rceil$. We denote $\sum_{i=j+2}^\ell r_i = 0$ if $j + 2 > \ell$.

7.2 Algebraic Attacks

The typical algebraic attacks on the BRD problem have been considered in [5, 40, 41]. The most powerful one is the Maximal Minors (MM) attack. From Table 6, the challenge of mounting this attack stems not just from its high time complexity, but from the fact that the required storage—at least a massive $2^{90} \times 2^{90}$ binary matrix—is astronomically large and infeasible to realize.

The MM Attack. By Equations (2) and (3), we have the equation $\varepsilon \mathbf{C} \mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$. This equation implies that $\mathbf{C} \mathbf{H}_y^\top \in \mathbb{F}_{q^m}^{r \times (n-k-1)}$ is not of row full rank because a non-zero vector ε belongs to its left kernel. Then all maximal minors $|\mathbf{C} \mathbf{H}_y^\top|_{*,J}$ of $\mathbf{C} \mathbf{H}_y^\top$ are equal to 0 for $J \subset \{1..n - k - 1\}$ and $\#J = r$. By the Cauchy-Binet formula, each $|\mathbf{C} \mathbf{H}_y^\top|_{*,J}$ can be viewed as a non-zero linear combination about all non-zero maximal minors $c_T = |\mathbf{C}|_{*,T}$ for $T \subset \{1..n\}$ and $\#T = r$. The number of non-zero maximal minors c_T 's is $\prod_{i=1}^\ell \binom{n_i}{r_i}$ and the number of non-zero linear combinations is at most $\binom{n-k-1}{r}$. One views non-zero c_T 's as unknowns and linear combinations as equations, and builds a linear system with $\prod_{i=1}^\ell \binom{n_i}{r_i}$ unknowns and at most $\binom{n-k-1}{r}$ equations over $\mathbb{F}_{q^m}$. This builds the MM modeling over $\mathbb{F}_{q^m}$:

$$\left\{ \left| \mathbf{C} \mathbf{H}_y^\top \right|_{*,J} = \sum_{T \subset \{1..n\}, \#T=r} c_T \left| \mathbf{H}_y^\top \right|_{T,J} = 0 : J \subset \{1..n - k - 1\}, \#J = r \right\}. \quad (11)$$

One wants to solve c_T 's from this linear system to determine $\mathbf{C}$. However, the system would have many solutions due to $\binom{n-k-1}{r} < \prod_{i=1}^\ell \binom{n_i}{r_i}$ whereas one wants more equations than unknowns for a unique solution. As $c_T \in \mathbb{F}_q$, to obtain more equations, one then unfolds the coefficients, namely $|\mathbf{H}_y^\top|_{T,J}$, over $\mathbb{F}_q$ and obtains at most $m \binom{n-k-1}{r}$ equations. In $\mathbb{F}_q$, the MM modeling includes:

Unknowns: $\prod_{i=1}^\ell \binom{n_i}{r_i}$ non-zero maximal minors $c_T \in \mathbb{F}_q$,

Equations: at most $m \binom{n-k-1}{r}$ equations over $\mathbb{F}_q$ in all non-zero c_T 's.

Once all non-zero c_T 's are solved from the MM modeling, one can determine the entries of $\mathbf{C}$. This point is associated with the Plücker coordinates in Grassmannian manifold (see Section 3 in [10] for a tailored description). Finally, one solves ε from the equation $\varepsilon \mathbf{C} \mathbf{H}_y^\top = \mathbf{0}_{n-k-1}$ and determines $\mathbf{e} = \varepsilon \mathbf{C}$.

The MM attack amounts to solving a linear system, whose unknowns and linearly independent equations determine the complexity of the attack. The $m \binom{n-k-1}{r}$ equations include many linearly dependent equations. The difficulty of analyzing the MM modeling for the BRD problem is how to estimate precisely the number of independent equations. In work [39], we derived a tight upper bound of independent equations in the MM modeling for general blocks. This bound is denoted by $\bar{E}(m, n, k, r, \ell, \mathbf{n}, \mathbf{r})$ and is given in Theorem 7.6.

Theorem 7.6. *For the blockwise errors with general ℓ , the number of linearly independent equations in the MM modeling over $\mathbb{F}_q$ is upper bounded by $\bar{E}(m, n, k, r, \ell, \mathbf{n}, \mathbf{r})$:*

$$\begin{cases} m \sum_{i=\ell-j+1}^{\ell} \sum_{\nu_i=\sum_{i=0}^{\ell-j} r_i - z_{\ell-j}}^{\ell-j} \binom{z_{\ell-j+1}}{r_{\ell-j+1} + z_{\ell-j} + \nu_{\ell-j+1}} \prod_{i=\ell-j+2}^{\ell} \binom{n_i}{r_i + \nu_i}, \\ \quad \text{if } \sum_{i=0}^{\ell-j} n_i < k+1 \leq \sum_{i=0}^{\ell-j} n_i + r_{\ell-j+1}; \\ m \sum_{i=\ell-j+1}^{\ell} \sum_{\nu_i=\sum_{i=0}^{\ell-j+1} r_i}^{\ell-j+1} \binom{z_{\ell-j+1}}{\nu_{\ell-j+1}} \prod_{i=\ell-j+2}^{\ell} \binom{n_i}{r_i + \nu_i}, \\ \quad \text{if } \sum_{i=0}^{\ell-j} n_i + r_{\ell-j+1} < k+1 \leq \sum_{i=0}^{\ell-j+1} n_i, \end{cases}$$

where $j \in \{1..\ell\}$, $z_{\iota} = \sum_{i=0}^{\iota} n_i - (k+1)$ for $\iota \in \{0..\ell\}$, $r_0 = 0$, and $n_0 = 0$. We denote $\prod_{i=\ell-j+2}^{\ell} \binom{n_i}{r_i + \nu_i} = 1$ if $\ell - j + 2 > \ell$.

One ensures $\bar{E}(m, n, k, r, \ell, \mathbf{n}, \mathbf{r}) \geq \prod_{i=1}^{\ell} \binom{n_i}{r_i} - 1$ such that the MM modeling has the rank $\prod_{i=1}^{\ell} \binom{n_i}{r_i} - 1$ for a unique solution.

For reducing the $\text{BRD}_{q,m,n,k,r}^{\ell, \mathbf{n}, \mathbf{r}}$ problem and obtaining the best complexity, the works [39, 41] proposed Blockwise Puncturing (BP), Blockwise Hybrid (BH), and Blockwise Subsupport Guessing (BSG) techniques. The BP technique reduces code length, while BH and BSG come at the cost of an expensive guessing step. Combinatorial attacks derive no benefit from these techniques, as they are either unaffected by code length or favor longer codes. In contrast, the MM attack is significantly improved by them. Let $\mathbf{p} = (p_1, p_2, \dots, p_{\ell})$, $\mathbf{a} = (a_1, a_2, \dots, a_{\ell})$, and $\mathbf{h} = (h_1, h_2, \dots, h_{\ell})$ be three integers vectors. Let $p = \sum_{i=1}^{\ell} p_i$, $a = \sum_{i=1}^{\ell} a_i$, and $h = \sum_{i=1}^{\ell} h_i$. Combining the BP, BH, and BSG techniques, solving $\text{BRD}_{q,m,n,k,r}^{\ell, \mathbf{n}, \mathbf{r}}$ is eventually equivalent to solving $\text{BRD}_{q,m,n-p-a-h,k-a,r-h}^{\ell, \mathbf{n}-\mathbf{p}-\mathbf{a}-\mathbf{h}, \mathbf{r}-\mathbf{h}}$ with a guessing cost $q^{\sum_{i=1}^{\ell} a_i(r_i - h_i) + h_i(n_i - h_i)}$, the overall complexity can be summarized as an integer nonlinear programming:

Theorem 7.7. *Let $\mathbf{p} = (p_1, p_2, \dots, p_{\ell})$, $\mathbf{a} = (a_1, a_2, \dots, a_{\ell})$, and $\mathbf{h} = (h_1, h_2, \dots, h_{\ell})$ be three integers vectors. Let $p = \sum_{i=1}^{\ell} p_i$, $a = \sum_{i=1}^{\ell} a_i$, and $h = \sum_{i=1}^{\ell} h_i$. With the BP, BH, and BSG techniques, the MM modeling solves $\text{BRD}_{q,m,n,k,r}^{\ell, \mathbf{n}, \mathbf{r}}$ with a time complexity*

$$\mathcal{O} \left(q^{\sum_{i=1}^{\ell} a_i(r_i - h_i) + h_i(n_i - h_i)} \bar{E} \left(\prod_{i=1}^{\ell} \binom{n_i - p_i - a_i - h_i}{r_i - h_i} \right)^{\omega-1} \right),$$

where $\bar{E} = \bar{E}(m, n - p - a - h, k - a, r, \ell, \mathbf{n} - \mathbf{p} - \mathbf{a} - \mathbf{h}, \mathbf{r} - \mathbf{h})$; $\mathbf{p}$, $\mathbf{a}$, and $\mathbf{h}$ such that $\bar{E} \geq \prod_{i=1}^{\ell} \binom{n_i - a_i - p_i - h_i}{r_i - h_i} - 1$ holds and the complexity is minimal; ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$.

This complexity is conservatively low because $\bar{E}$ is an upper bound of independent equations. A more precise complexity requires a tighter upper bound on independent equations. Nevertheless, the complexity difference should be very limited because our upper bound is tight.

7.3 Resistance to Quantum Attacks

Combinatorial Attacks. The overall complexity can be decomposed into a polynomial component and an exponential component. The polynomial part is dominated by solving a linear system, while the exponential part arises from guessing special subspaces of the ambient vector space or the support/coefficient matrix. In NIST PQC submissions RQC [30], it is commonly assumed that the guessing phase can be accelerated by a quadratic speedup using Grover’s algorithm. This corresponds to keeping the polynomial component unchanged while reducing the exponential component by a square-root factor. In this work, we adopt a more optimistic assumption that the polynomial component may also benefit from a quadratic speedup. Under this assumption, we estimate the quantum time complexity as the square root of the overall classical complexity.

Algebraic Attacks. The best algebraic attack is the MM attack, whose complexity primarily depends on solving a huge linear system and guessing certain entries of the coefficient matrix. The guessing phase can, in principle, be accelerated by a quadratic speedup using Grover’s algorithm. The resulting linear system is typically very large and exhibits approximately moderate density over finite fields. When the MM modeling is overdetermined, the overall complexity is dominated by the cost of solving this linear system, which is far from negligible. Moreover, unlike the regular structure of guessing operations, the irregular nature of linear system solving may prevent achieving a straightforward quadratic speedup. As noted in the NIST PQC submissions RYDE [2] and Mirath [1], such a square-root acceleration cannot be directly applied. To date, no specialized quantum algorithm is known for efficiently solving linear systems over finite fields.

- One candidate quantum approach is the Harrow–Hassidim–Lloyd (HHL) algorithm. However, HHL is designed for sparse systems over real or complex fields. Its complexity is given by $\mathcal{O}\left(\log N \cdot \frac{\tau^2 \kappa^2}{\varepsilon}\right)$, where N is the size of matrix, τ is sparsity, κ is condition number, and ε the precision of solutions ($0 \leq \varepsilon \leq 1$, where smaller values of ε correspond to higher precision). When attempting to apply this framework to finite fields, fundamental issues arise: the condition number κ is not well-defined, and exact solutions require $\varepsilon = 0$, which falls outside the algorithm’s assumptions.

Unlike Grover’s algorithm, which has been well established for accelerating guessing procedures, there is currently no concrete evidence supporting similar speedups for solving huge linear systems of approximately moderate density over finite fields. Nevertheless, given the current uncertainty surrounding quantum algorithms and hardware, we adopt an optimistic assumption that solving such linear systems may also benefit from a quadratic speedup. Under this assumption, we estimate the quantum time complexity as the square root of the overall classical complexity, in line with standard heuristics used for combinatorial attacks.

7.4 Estimated Security Strength

In this section, by combinatorial and algebraic attacks recalled in Sections 7.1 and 7.2, we estimate security strength for the parameter sets proposed in Table 2.

We first want to say that **our estimations are conservative.**

1. **The linear algebra constant ω is conservative and is set as 2.** All attacks against the RD problem involve solving a linear system. Typically, the most powerful MM algebraic attack, the complexity is completely determined by solving a linear system when

the MM modeling is overdetermined. Solving a linear system amounts to finding a vector belonging to the right kernel of its matrix. Recall that for a $\mu \times \nu$ matrix of rank ρ over $\mathbb{F}_q$, the cost to find a vector in its right kernel is $\mathcal{O}(\rho^{\omega-2}\mu\nu)$ operations in $\mathbb{F}_q$ [14, 24, 42], where ω is the matrix multiplication exponent (or linear algebra constant), $2 \leq \omega \leq 3$. The cost is determined by the linear algebra constant ω . The value ω usually is considered to be 3. Precisely, ω depends on the specific matrices and algorithms.

- The Strassen algorithm [44]. It is a *practical* algorithm for any matrix. If the matrix contains more than a few hundred rows and columns, a practical ω is 2.807.
- The Coppersmith-Winograd algorithm [17] and its improvements [23, 43, 46]. This type of algorithms is based on the laser method introduced by Strassen [45]. In [23], ω is optimized to about 2.373, which is currently the best known value. However, they are considered to be only *theoretical* and *impractical* due to at least an extremely large constant factor.
- The Wiedemann algorithm [49]. It is a *practical* algorithm for a *sparse* matrix. If the matrix only has τ non-zero terms per row, then the cost is given by $\mathcal{O}(\tau\mu\nu)$. If $\tau = \mathcal{O}(1)$, the cost is the same as one using $\omega = 2$.

Overall, it is widely held that the practical ω is about 2.807 and the theoretical lower bound approaches 2. However, the development of a practical algorithm that achieves the lower bound 2 while maintaining a tractable constant factor is a very *long-term* challenge in computational theory. At least, for a huge size or moderately dense matrix, a practical ω should still be 2.807. For the most powerful MM attack, the induced linear systems usually have a huge size and (approximately) moderate density. Nevertheless, for security parameters in Table 2, we conservatively set $\omega = 2$. This setting also takes into account any algorithm that could take advantage of the structure of the matrices.

2. **The classical time complexity of the MM attack against the BRD problem is conservative.** The used complexity formula [39] is given by a tight upper bound of independent equations. Generally, the more independent equations are, the lower the complexity is.
3. **The classical time complexity redundancy is conservative.** We reserve the complexity redundancy of at least 39 bits. We refer to Table 3, Table 6, and Table 7.
4. **The quantum time complexity is conservative.** As described in Section 7.3, we make more optimistic assumptions than the NIST PQC submissions [1, 2, 30] for the estimation of the quantum time complexity. We estimate the quantum time complexity as the square root of the classical time complexity (including solving a linear system).
5. **The space complexity is conservative.** While the MM attack leads to the lowest time complexity, it is difficult to devise such an attack because the induced linear system is very huge. It requires a very massive memory, about 2^{181} memory for the time complexity of 2^{181} . We only consider the space complexity as the memory required to store a linear system. We refer to Table 6.

In Tables 6 and 7, we compute the complexity of the best attacks on the BRD problems. In Table 6, we compute the (time and space) complexity of the MM algebraic attack with the BP approach [39], which is the best classical attack, and one can verify the complexity by the optimal $\mathbf{a}$, $\mathbf{p}$, and $\mathbf{h}$. In Table 7, we compute the classical time complexity of the combinatorial attacks such as the AGHT, PRR, and OJ attacks. “Attack-2n” represents attacking the

$\text{IBRSD}_{q,m,2n,n,w_{\mathbf{x}}+w_{\mathbf{y}}}^{2,(n,n),(w_{\mathbf{x}},w_{\mathbf{y}})}$ problem. “Attack-3n” represents attacking the $\text{IBRSD}_{q,m,3n,n,w_{\mathbf{r}_1}+w_{\mathbf{e}}+w_{\mathbf{r}_2}}^{3,(n,n,n),(w_{\mathbf{r}_1},w_{\mathbf{e}},w_{\mathbf{r}_2})}$ problem. The complexity of these attacks is recalled in Section 7.

Table 6: Classical time complexity of the **MM** algebraic attack

Instance	Attack-2n	Attack-3n	Minimal
	$(\xi, a_1, a_2, \mathbf{p}, \xi', \text{TC}, \text{SC})$	$(\xi, a_1, a_2, a_3, p_1, p_2, p_3, \xi', \text{TC}, \text{SC})$	TC
BRQC-128	$(0.37, 0, 22, \mathbf{0}, 1.05, 2^{216}, 2^{106})$	$(1.67, 0, 0, 0, 0, 0, 32, 1.01, 2^{181}, 2^{181})$	2^{181}
BRQC-256	$(0.27, 0, 37, \mathbf{0}, 1.02, 2^{309}, 2^{124})$	$(0.69, 1, 2, 7, 0, 0, 0, 1.00, 2^{295}, 2^{231})$	2^{295}
BRQC-512	$(0.10, 0, 72, \mathbf{0}, 1.04, 2^{584}, 2^{152})$	$(0.29, 0, 0, 36, 0, 0, 0, 1.03, 2^{566}, 2^{278})$	2^{566}

“ ξ ” (resp. “ ξ' ”) represents the (resp. updated) rate of equations and unknowns. “TC” represents the time complexity of the MM attack. “SC” represents the space complexity of the MM attack, which is computed as the memory required to store a linear system. We conservatively estimate “SC” as the size of a linear system. The optimal $\mathbf{h} = \mathbf{0}$.

Table 7: Classical time complexity of the **AGHT, PRR, OJ** combinatorial attacks

Instance	Attack-2n	Attack-3n	Minimal
	(AGHT, PRR, OJ)	(AGHT, PRR, OJ)	TC
BRQC-128	$(2^{550}, 2^{545}, 2^{506})$	$(2^{676}, 2^{624}, 2^{508})$	2^{506}
BRQC-256	$(2^{779}, 2^{698}, 2^{675})$	$(2^{1023}, 2^{1003}, 2^{840})$	2^{675}
BRQC-512	$(2^{1310}, 2^{1196}, 2^{1170})$	$(2^{1652}, 2^{1849}, 2^{1629})$	2^{1170}

We only compute the time complexity (TC). The space complexity is about the memory required to store a linear system, namely the polynomial part of the complexity formula. The space complexity is negligible to the MM attack.

8 Side-Channel Attacks

Currently, no tailored side-channel attack exists against BRQC. The side-channel attacks on RQC and the RD problem can be adapted to BRQC.

The resistance of RQC to timing attacks, as studied in [13], can be adapted to BRQC. These timing attacks exploit a correlation between the weight of the error to be decoded and the running time of the decoding algorithm. While of theoretical interest, such attacks are quite impractical in real-world scenarios as they require a huge number of queries to a timing oracle. Moreover, the reference and optimized implementations are designed to avoid leaking the error weight, thus preventing these timing attacks.

The works [18, 34] analyzed side-channel attacks on the RD problem, showing that partial leakage of information about the solution can weaken the problem’s hardness. This analysis can be adapted to the BRD problem. However, the critical premise lies in how such information about the solution can be obtained. In line with the timing attacks discussed above, our implementations are designed to avoid the leakage of information about the solution.

9 Advantages and Limitations

Advantages. The main advantages are:

- The security is only reduced to a well-understood problem in coding theory: the syndrome decoding problem.
- The security immunity against attacks recovering the hidden structure of a specific code.

- It features a small public key size.
- It features a null DFR.
- It features a clean, clear, and flexible structure.
- It features more attractive sizes than most of the schemes using the Hamming metric codes.

Limitations. The main limitations are:

- The rank metric for cryptosystems may still not be old (1991). Although this might be viewed as a limitation, there also exist further activities on understanding the inherent computational complexity of the related problems. Nevertheless, the combinatorial attacks have been very well understood, and the recent works [5, 39, 41] allow a clear view on the complexity of algebraic attacks, giving conservative estimations of complexity. Moreover, we have selected very conservative parameters for potential attacks.
- Unlike lattice-based schemes and the Ring Learning with Errors problem, the IBRD problem does not yet admit a known search-to-decision reduction.

References

- [1] Aragon, G.A., Aragon, N., Barbero, S., et al.: Mirath. Second round submission to the NIST post-quantum cryptography call: additional digital signature schemes (2025), <https://pqc-mirath.org/> 27, 28
- [2] Aragon, N., Bardet, M., Bidoux, L., et al.: RYDE. Second round submission to the NIST post-quantum cryptography call: additional digital signature schemes (2025), <https://pqc-ryde.org/> 27, 28
- [3] Aragon, N., Bettaieb, S., Bidoux, L., Connan, Y., Coulaud, J., Gaborit, P., Kominiaz, A.: The rank-based cryptography library. In: International Workshop on Code-Based Cryptography (CBCrypto). vol. 13150, pp. 22–41. Springer (2021) 16
- [4] Aragon, N., Blazy, O., Deneuville, J., Gaborit, P., Zémor, G.: Ouroboros: An efficient and provably secure KEM family. *IEEE Transactions on Information Theory* **68**(9), 6233–6244 (2022) 7
- [5] Aragon, N., Briaud, P., Dyseryn, V., Gaborit, P., Vinçotte, A.: The blockwise rank syndrome learning problem and its applications to cryptography. In: Post-Quantum Cryptography (PQCrypto). vol. 14771, pp. 75–106. Springer (2024) 20, 25, 30
- [6] Aragon, N., Gaborit, P., Hauteville, A., Ruatta, O., Zémor, G.: Low rank parity check codes: New decoding algorithms and applications to cryptography. *IEEE Transactions on Information Theory* **65**(12), 7697–7717 (2019) 7
- [7] Aragon, N., Gaborit, P., Hauteville, A., Tillich, J.: A new algorithm for solving the rank syndrome decoding problem. In: International Symposium on Information Theory (ISIT). pp. 2421–2425. IEEE (2018) 20, 21
- [8] Augot, D., Loidreau, P., Robert, G.: Generalized Gabidulin codes over fields of any characteristic. *Designs, Codes and Cryptography* **86**(8), 1807–1848 (2018) 11, 12
- [9] Bardet, M., Briaud, P., Bros, M., Gaborit, P., Neiger, V., Ruatta, O., Tillich, J.: An algebraic attack on rank metric code-based cryptosystems. In: Advances in Cryptology - EUROCRYPT. vol. 12107, pp. 64–93. Springer (2020) 3
- [10] Bardet, M., Briaud, P., Bros, M., Gaborit, P., Tillich, J.: Revisiting algebraic attacks on MinRank and on the rank decoding problem. *Designs, Codes and Cryptography* **91**(11), 3671–3707 (2023) 3, 25
- [11] Bardet, M., Bros, M., Cabarcas, D., Gaborit, P., Perlner, R.A., Smith-Tone, D., Tillich, J., Verbel, J.A.: Improvements of algebraic attacks for solving the rank decoding and MinRank problems. In: Advances in Cryptology - ASIACRYPT. vol. 12491, pp. 507–536. Springer (2020) 3
- [12] Berlekamp, E.R., McEliece, R.J., van Tilborg, H.C.A.: On the inherent intractability of certain coding problems (corresp.). *IEEE Transactions on Information Theory* **24**(3), 384–386 (1978) 5, 6
- [13] Bettaieb, S., Bidoux, L., Gaborit, P., Marcatel, E.: Preventing timing attacks against RQC using constant time decoding of gabidulin codes. In: International Conference on Post-Quantum Cryptography (PQCrypto). vol. 11505, pp. 371–386. Springer (2019) 29

- [14] Bunch, J.R., Hopcroft, J.E.: Triangular factorization and inversion by fast matrix multiplication. *Mathematics of Computation* **28**(125), 231–236 (1974) 28
- [15] Buss, J.F., Frandsen, G.S., Shallit, J.O.: The computational complexity of some problems of linear algebra. *Journal of Computer and System Sciences* **58**(3), 572–596 (1999) 5
- [16] Byrne, E., Gluesing-Luerssen, H., Ravagnani, A.: Fundamental properties of sum-rank-metric codes. *IEEE Transactions on Information Theory* **67**(10), 6456–6475 (2021) 3
- [17] Coppersmith, D., Winograd, S.: Matrix multiplication via arithmetic progressions. In: *Symposium on Theory of Computing (STOC)*. pp. 1–6. ACM (1987) 28
- [18] D’Alconzo, G., Esser, A., Gangemi, A., Sanna, C.: Sneaking up the ranks: Partial key exposure attacks on rank-based schemes. *Design, Codes and Cryptography* **94**(1), 15 (2026) 29
- [19] Faugère, J., Levy-dit-Vehel, F., Perret, L.: Cryptanalysis of MinRank. In: *Advances in Cryptology - CRYPTO*. vol. 5157, pp. 280–296. Springer (2008) 5, 6
- [20] Gabidulin, E.M.: Theory of codes with maximum rank distance. *Problemy peredachi informatsii* **21**(1), 3–16 (1985) 9, 11
- [21] Gaborit, P., Ruatta, O., Schrek, J.: On the complexity of the rank syndrome decoding problem. *IEEE Transactions on Information Theory* **62**(2), 1006–1019 (2016) 20, 21
- [22] Gaborit, P., Zémor, G.: On the hardness of the decoding and the minimum distance problems for rank codes. *IEEE Transactions on Information Theory* **62**(12), 7245–7252 (2016) 5
- [23] Gall, F.L.: Powers of tensors and fast matrix multiplication. In: *International Symposium on Symbolic and Algebraic Computation (ISSAC)*. pp. 296–303. ACM (2014) 28
- [24] von zur Gathen, J., Gerhard, J.: *Modern Computer Algebra*. Cambridge University Press (2013) 28
- [25] Glabush, L., Hövelmanns, K., Stebila, D.: On the multi-target security of post-quantum key encapsulation mechanisms. *IACR Communications in Cryptology* **3**(1), 20 (2026) 8, 14, 16, 19
- [26] Hauteville, A., Tillich, J.: New algorithms for decoding in the rank metric and an attack on the LRPC cryptosystem. In: *IEEE International Symposium on Information Theory (ISIT)*. pp. 2747–2751. IEEE (2015) 7, 15
- [27] Jerkovits, T., Bartz, H., Wachter-Zeh, A.: Support-guessing decoding algorithms in the sum-rank metric. *arXiv preprint* (2024), <https://arxiv.org/abs/2410.15806> 3
- [28] Loidreau, P.: A Welch-Berlekamp like algorithm for decoding Gabidulin codes. In: *International Workshop on Coding and Cryptography (WCC)*. vol. 3969, pp. 36–45. Springer (2005) 11, 12
- [29] Melchor, C.A., Aragon, N., Bardet, M., et al.: ROLLO. Second round submission to the NIST post-quantum cryptography call (2020), <https://pqc-rollo.org/> 7
- [30] Melchor, C.A., Aragon, N., Bettaieb, S., et al.: RQC. Second round submission to the NIST post-quantum cryptography call (2020), <http://pqc-rqc.org/> 3, 27, 28

- [31] da Nóbrega, R.W., Filho, B.F.U.: Multishot codes for network coding using rank-metric codes. In: IEEE International Workshop on Wireless Network Coding (2010) [3](#)
- [32] Ore, O.: On a special class of polynomials. Transactions of the American Mathematical Society **35**(3), 559–584 (1933) [9](#), [10](#)
- [33] Ourivski, A.V., Johansson, T.: New technique for decoding codes in the rank metric and its cryptography applications. Problems of Information Transmission **38**(3), 237–246 (2002) [23](#)
- [34] Porwal, A., Banda, H., Brinkmann, J., Baumeister, A., Krämer, J., Wachter-Zeh, A.: Subspace guessing and rank-metric solvers with hints. IACR Cryptology ePrint Archive p. 132 (2026), <https://eprint.iacr.org/2026/132> [29](#)
- [35] Puchinger, S., Renner, J., Rosenkilde, J.: Generic decoding in the sum-rank metric. IEEE Transactions on Information Theory **68**(8), 5075–5097 (2022) [3](#), [6](#), [21](#)
- [36] Richter, G., Plass, S.: Error and erasure decoding of rank-codes with a modified Berlekamp-Massey algorithm. In: Proceedings of ITG Conference on Source and Channel Coding. p. 203–2112 (2004) [11](#)
- [37] Richter, G., Plass, S.: Fast decoding of rank-codes with rank errors and column erasures. In: International Symposium on Information Theory (ISIT). p. 398. IEEE (2004) [11](#)
- [38] Song, Y., Chen, R., Huang, X., Zhang, J., Lin, C.: Refined OJ attacks: Tight complexity for rank decoding problems and their cryptographic implications. Cryptology ePrint Archive, Paper 2026/1291 (2026), <https://eprint.iacr.org/2026/1291> [23](#)
- [39] Song, Y., Chen, R., Zhang, F., Huang, X., Weng, J., Wang, H.: (Interleaved) extended Gabidulin codes, more analysis on blockwise rank decoding problem, and their applications to cryptosystems. Cryptology ePrint Archive, Paper 2025/668 (2025), <https://eprint.iacr.org/2025/668> [26](#), [28](#), [30](#)
- [40] Song, Y., Zhang, J., Huang, X., Wu, W.: Blockwise rank decoding problem and LRPC codes: Cryptosystems with smaller sizes. In: Advances in Cryptology - ASIACRYPT. vol. 14444, pp. 284–316. Springer (2023) [3](#), [5](#), [20](#), [23](#), [24](#), [25](#)
- [41] Song, Y., Zhang, J., Huang, X., Wu, W.: Blockwise rank decoding problem and LRPC codes: Cryptosystems with smaller sizes. IEEE Transactions on Information Theory **71**(6), 4806–4838 (2025) [3](#), [5](#), [6](#), [20](#), [21](#), [23](#), [24](#), [25](#), [26](#), [30](#)
- [42] Storjohann, A.: Algorithms for matrix canonical forms. Ph.D. thesis, ETH Zurich (2000) [28](#)
- [43] Stothers, A.J.: On the complexity of matrix multiplication. Ph.D. thesis, The University of Edinburgh (2010) [28](#)
- [44] Strassen, V.: Gaussian elimination is not optimal. Numerische Mathematik **13**(4), 354–356 (1969) [28](#)
- [45] Strassen, V.: The asymptotic spectrum of tensors and the exponent of matrix multiplication. In: Annual Symposium on Foundations of Computer Science (FOCS). pp. 49–54. IEEE Computer Society (1986) [28](#)

- [46] Vassilevska Williams, V.: Multiplying matrices faster than Coppersmith-Winograd. In: Symposium on Theory of Computing Conference (STOC). pp. 887–898. ACM (2012) [28](#)
- [47] Wachter, A., Sidorenko, V., Bossert, M.: A fast linearized Euclidean algorithm for decoding Gabidulin codes. In: Proceedings of ACCT. pp. 298–303 (2010) [11](#)
- [48] Wachter-Zeh, A., Afanassiev, V.B., Sidorenko, V.: Fast decoding of Gabidulin codes. Designs, Codes and Cryptography **66**(1-3), 57–73 (2013) [11](#)
- [49] Wiedemann, D.H.: Solving sparse linear equations over finite fields. IEEE Transactions on Information Theory **32**(1), 54–62 (1986) [28](#)